

Guida Pratica alla Teoria e agli Esercizi di Machine Learning

Spiegazioni, Soluzioni e Derivazioni Passo-Passo

Alessandro Gautieri

19 febbraio 2026

Indice

1	Machine Learning	5
1.1	Linear Algebra for Data Science: Vectors & Projections	5
1.1.1	1. Cosa è un Dato? (Vettori e Spazi)	5
1.1.2	2. Operazioni Fondamentali	5
1.1.3	3. Norme: Quanto è grande un vettore?	6
1.1.4	4. Il Prodotto Scalare (Dot Product)	6
1.1.5	5. Proiezioni Ortogonali	6
1.1.6	6. Basi e Cambio di Coordinate	7
1.2	Principal Component Analysis (PCA): La Geometria dei Dati	8
1.2.1	1. Fondamenti: Trasformazioni Lineari	8
1.2.2	2. Il Cambio di Base	8
1.2.3	3. L'Obiettivo della PCA	8
1.2.4	4. La Soluzione Matematica: Autovettori e Autovalori	9
1.2.5	5. Algoritmo PCA: La Ricetta Passo-Passo	9
1.2.6	6. Esercizio Svolto (Simulazione)	10
1.2.7	7. Approfondimento Teorico: Manifolds	10
1.3	Optimization: Il Motore dell'Apprendimento	11
1.3.1	1. Definizioni Fondamentali	11
1.3.2	2. Il Ruolo delle Derivate	11
1.3.3	3. Funzioni Quadratiche e Convessità	11
1.3.4	4. Gradient Descent: L'Algoritmo	12
1.3.5	5. Patologie dell'Ottimizzazione	12
1.3.6	6. Gradient Descent per la PCA	12
1.3.7	7. Esercizio Svolto: Discesa Manuale	13
1.4	Modelli Lineari e Valutazione: Le Fondamenta del Supervised Learning . .	14
1.4.1	k-Nearest Neighbors (k-NN)	14
1.4.2	1. Il Framework dell'Apprendimento	15
1.4.3	2. Regressione Lineare	15
1.4.4	3. Regressione Logistica (Classificazione)	16
1.4.5	4. Valutazione: Oltre l'Accuratezza	16
1.5	Alberi di Decisione ed Ensemble Methods	18
1.5.1	1. Parametric vs Non-Parametric	18
1.5.2	2. Decision Trees (Alberi di Decisione)	18
1.5.3	3. Ensemble Methods: L'Unione fa la Forza	19
1.5.4	4. Riassunto Finale	20
2	Deep Learning	21

2.1	Dalle Feature Manuali alle Reti Neurali (Shallow Networks)	21
2.1.1	1. Il Cambio di Paradigma: Representation Learning	21
2.1.2	2. Il Perceptron (Il Neurone Singolo)	21
2.1.3	3. Esercizio Svolto: Il Problema dello XOR	22
2.1.4	4. Multilayer Perceptron (MLP) e Funzioni di Attivazione	22
2.2	Forward e Backward Propagation: Il Motore del Learning	24
2.2.1	1. Forward Propagation (Il flusso in avanti)	24
2.2.2	2. La Loss Function (Come misuriamo l'errore)	24
2.2.3	3. Backpropagation (L'Algoritmo di Apprendimento)	25
2.3	Deep Neural Networks: Architetture e Ottimizzazione Avanzata	27
2.3.1	1. L'Intuizione Geometrica: La "Piegatura" dello Spazio	27
2.3.2	2. Formulazione Matematica di una Rete Profonda	27
2.3.3	3. Le Sfide dell'Ottimizzazione	27
2.3.4	4. Evoluzione degli Algoritmi di Ottimizzazione	28
2.3.5	5. Il Problema del Vanishing Gradient e le Soluzioni	29
2.3.6	6. Residual Networks (ResNet)	29
2.3.7	7. Batch Normalization (BN)	30
2.3.8	8. Regolarizzazione: Dropout	30
2.4	Self-Supervised Learning e Masked Modeling	31
2.4.1	Masked Signal Modeling	31
2.4.2	Masked Autoencoder (MAE)	31
2.5	Image Filtering: I Fondamenti della Visione Artificiale	33
2.5.1	1. L'Immagine come Funzione	33
2.5.2	2. Il Filtraggio (Smoothing)	33
2.6	Architectures for Grids: Convolutional Neural Networks (CNN)	35
2.6.1	1. Perché le Reti Fully Connected (MLP) falliscono sulle immagini?	35
2.6.2	2. La Soluzione: Convolutional Neural Networks (CNN)	35
2.6.3	3. Anatomia di un Layer Convoluzionale	35
2.6.4	4. Iperparametri Chiave: Stride, Padding, Dilation	35
2.6.5	5. Pooling: Invarianza e Compressione	36
2.6.6	6. Architetture Famose (Evoluzione Storica)	36
2.6.7	7. Receptive Field (Campo Ricettivo)	36
2.7	Memory and Sequence Modeling: Gestire il Tempo	37
2.7.1	1. Perché il Tempo è Speciale?	37
2.7.2	2. Primo Approccio: Convoluzioni Causali (1D CNN)	37
2.7.3	3. La Soluzione Elegante: Recurrent Neural Networks (RNN)	37
2.7.4	4. Deep RNN e Unrolling	38
2.7.5	5. Backpropagation Through Time (BPTT)	38
2.7.6	6. Sequence Modeling e NLP (Natural Language Processing)	39
2.7.7	7. Confronto Finale: Chi gestisce meglio la Memoria?	39
2.8	Transformers: La Rivoluzione dell'Attenzione	40
2.8.1	1. I Limiti dei Predecessori	40
2.8.2	2. Innovazione 1: I Token (L'atomo del dato)	40
2.8.3	3. Innovazione 2: L'Attenzione (Self-Attention)	40
2.8.4	4. Innovazione 3: Positional Encoding	42
2.8.5	5. L'Architettura Completa (Encoder-Decoder)	42
2.8.6	6. Vision Transformers (ViT)	42
2.9	Introduzione ai Modelli Generativi: L'Arte della Creazione	43

2.9.1	1. Discriminative vs Generative Modeling	43
2.9.2	2. Conditional Generative Models	43
2.9.3	3. La Tassonomia dei Modelli Generativi	43
2.9.4	4. Autoregressive Models (PixelRNN / PixelCNN)	44
2.9.5	5. Uno Sguardo al Futuro: VAE e Diffusion	44
2.10	Modelli Generativi Avanzati: VAE e GAN	46
2.10.1	1. Latent Variable Models (Il concetto di z)	46
2.10.2	2. Variational Autoencoders (VAE)	46
2.10.3	3. Generative Adversarial Networks (GAN)	47
2.10.4	4. Confronto Finale: VAE vs GAN	48
2.11	Le Funzioni di Costo (Loss Functions)	49
2.11.1	1. Regressione (Predizione di valori continui)	49
2.11.2	2. Classificazione Binaria (Due classi: 0 o 1)	49
2.11.3	3. Classificazione Multi-classe (K classi)	50
2.11.4	4. Modelli Generativi (es. VAE)	50
3	Ripasso Rapido	53
3.1	Concetti Fondamentali: La Chain Rule	53
3.1.1	L'Analogia del Domino	53
3.1.2	L'Analogia della Cipolla (Funzioni Annidate)	53
3.2	Approfondimenti Intuitivi sulle Reti Neurali	54
3.2.1	1. Perché usiamo le Funzioni di Attivazione (Non-Linearità)?	54
3.2.2	2. Pesi (W) vs Bias (b): Cosa fanno davvero?	54
3.2.3	3. Il Learning Rate (α): L'Escursionista nella Nebbia	54
4	Esercizi	56
4.1	Esercizio Svolto di PCA (Guida Passo-Passo)	56
4.1.1	1. Il Dataset (Esempio)	56
4.1.2	2. Centrazione dei Dati (Step Fondamentale)	56
4.1.3	3. Matrice di Covarianza	56
4.1.4	4. Autovalori e Autovettori	57
4.1.5	5. Varianza Spiegata (Domanda 7)	57
4.1.6	6. Proiezione e Ricostruzione	57
4.1.7	7. MSE (Mean Squared Reconstruction Error)	58
4.2	Esercizio: Backpropagation su Shallow Neural Network	58
4.2.1	Il Modello (Forward Pass)	58
4.2.2	Backward Pass: Calcolo dei Gradienti	58
4.3	Esercizi di Ottimizzazione: Spiegazione Dettagliata	60
4.3.1	Esercizio 1: Forward Mode AD (Differenziazione Automatica in Avanti)	60
4.3.2	Esercizio 2: Reverse Mode AD (Backpropagation Manuale)	60
4.3.3	Esercizio 3: Backpropagation con Tanh (Funzione di Attivazione) .	61
4.3.4	Esercizio 5: Gradient Descent (1 Variabile)	62
4.3.5	Esercizio 6: Gradient Descent (2 Variabili)	63
4.4	Esercizi Svolti: Regressione e Classificazione	64
4.4.1	Esercizio 1: Regressione Lineare (Equazioni Normali)	64
4.4.2	Esercizio 2: Regressione Lineare (Gradient Descent)	65
4.4.3	Esercizio 3: Binary Logistic Regression (Gradient Descent)	66
4.4.4	Esercizio 4: Multi-Class Logistic Regression (Softmax)	67

4.4.5	Esercizio 5: ROC Curve e AUC	68
4.5	Algoritmi Classici: Alberi, KNN, Random Forest	70
4.5.1	Albero Decisionale	70
4.5.2	k-Nearest Neighbors (k-NN)	70
4.5.3	Random Forest	70
4.6	Parte 1: Convoluzione 1D	71
4.7	Parte 2: Convoluzione 2D	72
4.8	Parte 3: Backpropagation (Rete Neurale)	73
4.8.1	1. Setup della Rete	73
4.8.2	2. Forward Pass (Calcolo delle Previsioni)	74
4.8.3	3. Backward Pass (Calcolo dei Gradienti)	74
4.8.4	4. Calcolo Gradienti dei Pesi e Aggiornamento	75

Capitolo 1

Machine Learning

1.1 Linear Algebra for Data Science: Vectors & Projections

Riferimento: FDS02-vectors&projections.pdf

Benvenuti alle fondamenta. Se il Deep Learning è un grattacielo, l'Algebra Lineare è il cemento armato. Senza capire vettori, spazi e proiezioni, tutto il resto sarà solo "magia nera". In questo modulo impariamo a parlare la lingua dei dati.

1.1.1 1. Cosa è un Dato? (Vettori e Spazi)

Nel nostro mondo, un dato non è un numero astratto. È un punto in uno spazio geometrico.

- **Scalare** ($x \in \mathbb{R}$): Un singolo numero (es. temperatura).
- **Vettore** ($\mathbf{x} \in \mathbb{R}^n$): Una lista ordinata di n numeri. *Esempio:* Nel dataset Iris, ogni fiore è un vettore $\mathbf{x} \in \mathbb{R}^4$ (lunghezza sepalo, larghezza sepalo, lunghezza petalo, larghezza petalo).
- **Interpretazione Geometrica:** Un vettore è una freccia che parte dall'origine $(0, 0, \dots)$ e arriva al punto definito dalle coordinate.

1.1.2 2. Operazioni Fondamentali

Le due operazioni che definiscono uno "Spazio Vettoriale" sono:

1. **Somma:** $\mathbf{z} = \mathbf{x} + \mathbf{y}$. Geometricamente, seguiamo la regola del parallelogramma (o "punta-coda").
2. **Moltiplicazione per Scalare:** $\mathbf{z} = \alpha \mathbf{x}$. Allunghiamo o accorciamo la freccia. Se $\alpha < 0$, la giriamo.

1.1.3 3. Norme: Quanto è grande un vettore?

Spesso dobbiamo misurare la "grandezza" di un vettore o la distanza tra due punti (errore). La ****Norma**** ($||\mathbf{x}||$) è la funzione che fa questo.

- **Norma L_2 (Euclidea):** La distanza classica "in linea d'aria".

$$||\mathbf{x}||_2 = \sqrt{\sum x_i^2}$$

Uso: È la norma standard per la regressione (MSE) e per la fisica.

- **Norma L_1 (Manhattan):** La distanza "del tassista". In una città a scacchiera, non puoi andare in diagonale. Devi sommare i blocchi orizzontali e verticali.

$$||\mathbf{x}||_1 = \sum |x_i|$$

Uso: Fondamentale nel Machine Learning per la ****Sparsità**** (Lasso Regularization). Tende a mandare molti pesi a zero.

- **Norma L_∞ (Max Norm):** Considera solo la componente più grande.

$$||\mathbf{x}||_\infty = \max_i |x_i|$$

1.1.4 4. Il Prodotto Scalare (Dot Product)

Questa è l'operazione più importante di tutto il corso. Il prodotto scalare tra due vettori $\mathbf{x}$ e $\mathbf{y}$ ci dice ****quanto sono allineati****.

Definizione Algebrica

$$\mathbf{x} \cdot \mathbf{y} = \mathbf{x}^T \mathbf{y} = \sum_{i=1}^n x_i y_i$$

È la somma dei prodotti elemento per elemento.

Definizione Geometrica (L'Intuizione)

$$\mathbf{x} \cdot \mathbf{y} = ||\mathbf{x}|| ||\mathbf{y}|| \cos(\theta)$$

Dove θ è l'angolo tra i due vettori.

- Se $\mathbf{x} \cdot \mathbf{y} > 0$: L'angolo è acuto ($< 90^\circ$). Vanno nella stessa direzione.
- Se $\mathbf{x} \cdot \mathbf{y} < 0$: L'angolo è ottuso ($> 90^\circ$). Vanno in direzioni opposte.
- **Caso Critico:** Se $\mathbf{x} \cdot \mathbf{y} = 0$, i vettori sono ****Ortogonalità**** (perpendicolari). Non hanno nulla in comune.

1.1.5 5. Proiezioni Ortogonali

Spesso vogliamo "schiacciare" un vettore $\mathbf{y}$ su una retta (o sottospazio) definita da un vettore $\mathbf{a}$. Immagina di proiettare l'ombra di $\mathbf{y}$ su $\mathbf{a}$ quando il sole è a picco (perpendicolare).

La proiezione $\hat{\mathbf{y}}$ è il punto sulla retta $\mathbf{a}$ più vicino a $\mathbf{y}$. La formula deriva dal fatto che l'errore ($\mathbf{e} = \mathbf{y} - \hat{\mathbf{y}}$) deve essere ortogonale ad $\mathbf{a}$.

$$\text{Proj}_{\mathbf{a}}(\mathbf{y}) = \frac{\mathbf{y}^T \mathbf{a}}{\mathbf{a}^T \mathbf{a}} \mathbf{a}$$

Analisi:

- $\frac{\mathbf{y}^T \mathbf{a}}{\mathbf{a}^T \mathbf{a}}$ è uno scalare (un numero). Ci dice "quante volte" il vettore $\mathbf{a}$ sta dentro l'ombra di $\mathbf{y}$.
- Moltiplicando per $\mathbf{a}$ trasformiamo quel numero in un vettore lungo la direzione giusta.

1.1.6 6. Basi e Cambio di Coordinate

Un **Basis Set** è un insieme di vettori $\{v_1, \dots, v_n\}$ che possiamo usare per costruire qualsiasi altro vettore nello spazio tramite somma pesata.

$$\mathbf{x} = c_1 \mathbf{v}_1 + c_2 \mathbf{v}_2 + \dots + c_n \mathbf{v}_n$$

I coefficienti c_i sono le **coordinate** di $\mathbf{x}$ rispetto a quella base.

Perché cambiare base? Spesso i dati sono "storti" rispetto agli assi standard (x, y). Cambiare base (es. PCA - Principal Component Analysis) ci permette di trovare assi più significativi, dove i dati sono decorrelati o più facili da separare.

1.2 Principal Component Analysis (PCA): La Geometria dei Dati

Riferimento: FDS03-PCA.pdf

La PCA non è solo un algoritmo; è un modo di guardare i dati. Spesso lavoriamo con dati ad alta dimensionalità (es. immagini con migliaia di pixel), ma l'informazione utile risiede in uno spazio molto più piccolo. La PCA ci permette di trovare questo "sottospazio" ottimale ruotando i nostri assi per allinearli con le direzioni di massima variazione dei dati.

1.2.1 1. Fondamenti: Trasformazioni Lineari

Prima di capire la PCA, dobbiamo capire cosa fa una matrice A a un vettore x . L'operazione $y = Ax$ trasforma lo spazio:

- **Scaling:** Se A è diagonale, allunga o accorcia gli assi.
- **Rotazione/Riflessione:** Se A è ortonormale ($A^T A = I$), ruota lo spazio senza deformarlo.
- **Shearing (Taglio):** Se ci sono elementi fuori diagonale, gli assi si inclinano.
- **Proiezione:** Se il rango di A è minore della dimensione n , lo spazio collassa (es. da 3D a 2D). La PCA è, essenzialmente, una proiezione intelligente.

1.2.2 2. Il Cambio di Base

Immaginiamo di avere una nuova base ortonormale C (una matrice dove le colonne sono i nuovi vettori base).

- **Encoding (Proiezione):** Per passare dal vecchio spazio al nuovo, calcoliamo i "pesi" w :

$$w = C^T x$$

- **Decoding (Ricostruzione):** Per tornare indietro, usiamo i pesi per combinare i vettori base:

$$\hat{x} = Cw$$

Se usiamo **tutti** i vettori della base ($K = N$), la ricostruzione è perfetta ($\hat{x} = x$). Se usiamo solo **alcuni** vettori ($K < N$), stiamo comprimendo i dati. La PCA ci dice quali vettori scegliere per perdere meno informazioni possibile.

1.2.3 3. L'Obiettivo della PCA

Vogliamo trovare una direzione u (un vettore unitario) su cui proiettare i dati. Qual è la direzione migliore? Abbiamo due modi equivalenti di vedere il problema:

Visione 1: Massimizzare la Varianza (Max Variance)

Vogliamo che, una volta proiettati i punti sulla retta u , essi siano il più "sparpagliati" possibile. *Intuizione:* Se i punti sono tutti schiacciati in un punto, abbiamo perso informazione. Se sono ben distribuiti, l'abbiamo mantenuta.

$$\max_u \text{Var}(u^T X)$$

Visione 2: Minimizzare l'Errore di Ricostruzione (Min MSE)

Vogliamo che la distanza tra i punti originali x e la loro proiezione sulla retta $\hat{x}$ sia minima. *Intuizione:* La "somma dei quadrati delle distanze" (residui) deve essere la più piccola possibile.

Risultato Fondamentale: Le due visioni sono matematicamente identiche. La direzione che massimizza la varianza è la stessa che minimizza l'errore quadratico.

1.2.4 4. La Soluzione Matematica: Autovettori e Autovalori

Come troviamo queste direzioni magiche? Guardando la **Matrice di Covarianza** dei dati. Sia X la matrice dei dati (centrata, cioè con media zero). La matrice di covarianza è:

$$\Sigma = \frac{1}{N} X^T X$$

Questa matrice ci dice come le feature variano insieme. Le direzioni principali (Principal Components) sono semplicemente gli **Autovettori** di Σ .

- **Autovettori** (v_i): Le direzioni degli assi della PCA.
- **Autovalori** (λ_i): La quantità di varianza spiegata da quell'asse.

Ordiniamo gli autovalori $\lambda_1 > \lambda_2 > \dots$ per sapere quali assi sono più importanti.

1.2.5 5. Algoritmo PCA: La Ricetta Passo-Passo

Ecco la procedura standard da seguire negli esercizi:

1. **Centratura:** Calcola la media μ di ogni feature e sottraila ai dati.

$$X_{centered} = X - \mu$$

2. **Covarianza:** Calcola $\Sigma = \frac{1}{N-1} X_{centered}^T X_{centered}$.
3. **Eigendecomposition:** Trova λ_i, v_i tali che $\Sigma v_i = \lambda_i v_i$.
4. **Selezione:** Scegli i primi K autovettori corrispondenti ai λ più grandi. Questi formano la matrice di proiezione W_K .
5. **Proiezione:** I nuovi dati ridotti sono $Z = X_{centered} W_K$.

1.2.6 6. Esercizio Svolto (Simulazione)

Dati: Immaginiamo 3 punti 2D: $A(1, 2), B(2, 3), C(3, 4)$. **Obiettivo:** Calcolare la prima Componente Principale.

Step 1: Centratura Media $\mu_x = (1 + 2 + 3)/3 = 2$, $\mu_y = (2 + 3 + 4)/3 = 3$. Dati centrati: $A'(-1, -1), B'(0, 0), C'(1, 1)$.

Step 2: Matrice di Covarianza

$$\Sigma = \frac{1}{3-1} \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} \begin{bmatrix} -1 & -1 \\ 0 & 0 \\ 1 & 1 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 2 & 2 \\ 2 & 2 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

Step 3: Autovalori e Autovettori Risolviamo $\det(\Sigma - \lambda I) = 0$: $(1 - \lambda)(1 - \lambda) - 1 = 0 \Rightarrow (1 - \lambda)^2 = 1 \Rightarrow 1 - \lambda = \pm 1$. $\lambda_1 = 2, \lambda_2 = 0$. Per $\lambda_1 = 2$: Cerchiamo vettore $[x, y]$ tale che $\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = 2 \begin{bmatrix} x \\ y \end{bmatrix}$. $x + y = 2x \Rightarrow y = x$. Autovettore normalizzato: $v_1 = [\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}] \approx [0.707, 0.707]$.

Interpretazione: I dati sono perfettamente allineati sulla diagonale $y = x$. La PCA ha trovato questa retta come asse principale. La varianza lungo questo asse è 2. Lungo l'asse ortogonale è 0 (i punti sono allineati, non hanno "spessore").

1.2.7 7. Approfondimento Teorico: Manifolds

Le slide si concludono con un concetto avanzato. Perché la PCA funziona? Perché spesso i dati ad alta dimensionalità non riempiono tutto lo spazio, ma giacciono su una superficie curva a bassa dimensionalità chiamata **Manifold**.

- **Esempio:** Immagina un foglio di carta (2D) piegato a forma di 'U' nello spazio 3D. Anche se i punti hanno 3 coordinate (x, y, z) , la loro struttura intrinseca è 2D.
- **Charts Embedding:** Matematicamente, usiamo delle mappe ("Charts") per appiattare localmente il manifold in uno spazio Euclideo (proprio come un atlante mappa la Terra sferica su pagine piatte).

La PCA trova il miglior "iperpiano lineare" che approssima questo manifold.

1.3 Optimization: Il Motore dell'Apprendimento

Riferimento: FDS04-optimization.pdf

L'Ottimizzazione è l'atto di cercare il meglio. Nel Machine Learning, questo significa trovare i parametri w che minimizzano l'errore o massimizzano la verosimiglianza. Tutto il Deep Learning moderno si regge su una semplice idea: seguire la pendenza della montagna per arrivare a valle.

1.3.1 1. Definizioni Fondamentali

Un problema di ottimizzazione si scrive come:

$$w^* = \arg \min_w g(w)$$

Dove:

- $g(w)$: è la **Funzione Obiettivo** (o Loss Function se stiamo minimizzando l'errore).
- w : sono i **Parametri** (pesi) che possiamo modificare.
- w^* : è la configurazione ottimale (Global Minimum).

Nota: Minimizzare $g(w)$ è identico a massimizzare $-g(w)$. I concetti sono speculari.

1.3.2 2. Il Ruolo delle Derivate

Come troviamo il minimo?

1. **Analisi (Metodo Chiuso):** Cerchiamo i punti dove la derivata è zero ($\nabla g(w) = 0$).
 - Se $g''(w) > 0$ (convessa), è un minimo.
 - Se $g''(w) < 0$ (concava), è un massimo.
 - Se $g''(w) = 0$, potrebbe essere un *Punto di Sella* (né l'uno né l'altro, come il centro di una sella da cavallo).
2. **Metodo Iterativo (Gradient Descent):** Usato quando non possiamo risolvere l'equazione $\nabla g = 0$ direttamente (cioè quasi sempre nel Deep Learning).

1.3.3 3. Funzioni Quadratiche e Convessità

Le funzioni più "amichevoli" sono quelle **Convesse** (a forma di ciotola). Esempio classico: $g(w) = w^2$.

- Hanno un **unico** minimo globale.
- Non ci sono minimi locali in cui incastrarsi.
- La discesa del gradiente è garantita convergere al punto ottimo.

Purtroppo, le Reti Neurali sono funzioni **Non-Convesse**: hanno montagne, valli, buche locali e punti piatti. Trovare il minimo globale è difficile.

1.3.4 4. Gradient Descent: L'Algoritmo

Immagina di essere un escursionista cieco su una montagna. Vuoi scendere a valle. Cosa fai? Tasti il terreno con i piedi per sentire la pendenza (Gradiente) e fai un passo nella direzione in cui il terreno scende più ripido.

Regola di Aggiornamento:

$$w_{k+1} = w_k - \alpha \nabla g(w_k)$$

- $\nabla g(w_k)$: Il **Gradiente**. Punta verso la salita più ripida. Noi andiamo in direzione opposta (segno meno).
- α (Alpha): Il **Learning Rate** (o Step Size). Quanto è lungo il tuo passo.

Il Learning Rate (α)

È l'iperparametro più importante.

- α **troppo piccolo**: Scendi a passi di formica. Ci metti un'eternità a convergere.
- α **troppo grande**: Fai passi da gigante. Salti oltre la valle e finisci sul versante opposto, magari più in alto di prima. L'algoritmo *diverge* (esplode).
- α **giusto**: Passi decisi all'inizio, più piccoli man mano che la pendenza diminuisce (perché il gradiente stesso diventa più piccolo vicino al minimo).

1.3.5 5. Patologie dell'Ottimizzazione

Le slide evidenziano problemi critici nelle funzioni non-convesse:

1. **Minimi Locali**: Buche poco profonde dove l'algoritmo si ferma credendo di essere arrivato, ma il vero minimo globale è altrove.
2. **Punti di Sella (Saddle Points)**: Zone dove il gradiente è zero, ma non siamo né in cima né in fondo (es. w^3 a $w = 0$). In alte dimensioni, i punti di sella sono molto più frequenti dei minimi locali.
3. **Plateau (Pianure)**: Zone dove il gradiente è quasi zero (es. $g(w) = w^4 + 0.1$ vicino a zero). La discesa diventa lentissima ("Slow Crawling").

1.3.6 6. Gradient Descent per la PCA

Un'applicazione interessante (citata nelle slide) è usare il GD per calcolare la PCA senza dover fare la decomposizione spettrale completa (costosa). **Obiettivo**: Massimizzare la varianza proiettata.

$$\max_C \text{Tr}(C^T X X^T C) \quad \text{s.t. } C^T C = I$$

Possiamo muovere la matrice C seguendo il gradiente di questa funzione. È utile quando i dati sono così tanti che non possiamo calcolare la matrice di covarianza intera.

1.3.7 7. Esercizio Svolto: Discesa Manuale

Funzione: $g(w) = w^2 + 2w + 1$ (una parabola con minimo a $w = -1$). **Punto iniziale:** $w_0 = 3$. **Learning Rate:** $\alpha = 0.1$.

Obiettivo: Eseguire 2 passi di Gradient Descent.

Passo 0 (Inizializzazione): $w_0 = 3$. Calcoliamo il gradiente (derivata): $g'(w) = 2w + 2$. $g'(3) = 2(3) + 2 = 8$. (Pendenza molto ripida positiva).

Passo 1 (Update):

$$w_1 = w_0 - \alpha \cdot g'(w_0)$$

$$w_1 = 3 - 0.1 \cdot 8 = 3 - 0.8 = \mathbf{2.2}$$

Osservazione: Ci siamo spostati da 3 verso sinistra (2.2), avvicinandoci al minimo (-1).

Passo 2 (Update): Calcoliamo il nuovo gradiente in $w_1 = 2.2$: $g'(2.2) = 2(2.2) + 2 = 4.4 + 2 = 6.4$. (La pendenza è diminuita).

$$w_2 = w_1 - \alpha \cdot g'(w_1)$$

$$w_2 = 2.2 - 0.1 \cdot 6.4 = 2.2 - 0.64 = \mathbf{1.56}$$

Conclusione: In due passi siamo scesi da $3 \rightarrow 2.2 \rightarrow 1.56$. Stiamo convergendo verso il minimo (-1) rallentando progressivamente. Se avessimo usato $\alpha = 1.0$, saremmo saltati a $3 - 8 = -5$ (oltrepassando il minimo e allontanandoci).

1.4 Modelli Lineari e Valutazione: Le Fondamenta del Supervised Learning

Riferimento: *FDS05-linear_models.pdf*

In questo capitolo formalizziamo il problema dell'apprendimento. Il Machine Learning non è magia: è **Approssimazione di Funzioni**. Esiste una funzione ideale $f^*(x)$ che mappa perfettamente gli input agli output (es. i sintomi alla malattia), ma noi non la conosciamo. Il nostro obiettivo è trovare una funzione $f(x)$ nella nostra "famiglia di modelli" $\mathcal{F}$ che sia la migliore approssimazione possibile di f^* .

1.4.1 k-Nearest Neighbors (k-NN)

Il principio di prossimità: "Dimmi con chi vai e ti dirò chi sei."

Il k-NN è uno degli algoritmi più semplici e intuitivi del Machine Learning. È un metodo **non-parametrico** e **Lazy Learning** (apprendimento pigro): non costruisce un modello esplicito durante il training (nessun peso w da imparare), ma si limita a memorizzare l'intero dataset. Il calcolo vero e proprio avviene solo al momento dell'inferenza.

1. Funzionamento dell'Algoritmo

Dato un nuovo punto di input x_{new} che vogliamo classificare:

1. **Calcolo Distanze:** Misuriamo la distanza tra x_{new} e *tutti* gli esempi memorizzati nel training set.
2. **Selezione Vicini:** Identifichiamo i k punti più vicini (Nearest Neighbors).
3. **Votazione (Majority Vote):**
 - **Classificazione:** Assegniamo a x_{new} la classe più frequente tra i k vicini.
 - **Regressione:** Assegniamo a x_{new} la media dei valori dei k vicini.

2. Metriche di Distanza

La scelta della metrica è cruciale e dipende dalla geometria dei dati:

- **Distanza Euclidea (L_2):** La più comune. È la linea d'aria tra due punti.

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

- **Distanza Manhattan (L_1):** Somma delle differenze assolute (utile in spazi ad alta dimensionalità o griglie).

$$d(x, y) = \sum_{i=1}^n |x_i - y_i|$$

3. L'Iperparametro k e il Bias-Variance Tradeoff

Il valore di k controlla drasticamente la complessità del modello:

- k **Piccolo** (es. $k = 1$): Il modello è molto flessibile ma instabile. Tende a seguire il rumore dei dati. *Risultato*: Basso Bias, **Alta Varianza** (Overfitting).
- k **Grande** (es. $k = 100$): Il modello diventa rigido e tende a predire sempre la classe di maggioranza globale, ignorando i dettagli locali. *Risultato*: Alto Bias, **Bassa Varianza** (Underfitting).

4. Il Problema della Scala (Feature Scaling)

Il k-NN è estremamente sensibile all'ordine di grandezza delle feature. *Esempio*: Se classifichiamo persone usando "Età" (0-100) e "Reddito" (0-100.000), la distanza sarà dominata quasi interamente dal reddito, rendendo l'età irrilevante. **Soluzione Obbligatoria**: È fondamentale **normalizzare** (Min-Max Scaling) o **standardizzare** (Z-Score) i dati prima di applicare il k-NN, portando tutte le feature su una scala comparabile (es. tra 0 e 1).

1.4.2 1. Il Framework dell'Apprendimento

- **Hypothesis Space ($\mathcal{F}$)**: L'insieme di tutte le funzioni che il nostro modello può imparare (es. tutte le rette possibili).
- **Loss Function ($\mathcal{L}$)**: Una metrica che ci dice quanto la nostra predizione $f(x)$ è lontana dalla verità y .
- **Risk Minimization**: Idealmente vorremmo minimizzare l'errore su *tutti i dati possibili dell'universo* (Expected Risk). Poiché non li abbiamo, minimizziamo l'errore sul *dataset di training* (Empirical Risk Minimization - ERM).

1.4.3 2. Regressione Lineare

Il modello più semplice: assumiamo che l'output sia una combinazione lineare degli input.

$$\hat{y} = w^T x + b$$

Dove w sono i pesi (l'importanza di ogni feature) e b è il bias.

Come troviamo w ottimale?

Usiamo la Loss **MSE (Mean Squared Error)**:

$$J(w) = \frac{1}{N} \sum_{i=1}^N (y_i - w^T x_i)^2$$

Abbiamo due modi per trovare il minimo:

1. **Gradient Descent**: Metodo iterativo (visto nel capitolo precedente).

2. **Soluzione Analitica (Equazioni Normali):** Possiamo calcolare la soluzione esatta in un colpo solo azzerando la derivata!

$$\hat{w} = (X^T X)^{-1} X^T Y$$

Intuizione Geometrica: Stiamo proiettando il vettore Y sullo spazio generato dalle colonne di X . L'errore è ortogonale al piano dei dati.

1.4.4 3. Regressione Logistica (Classificazione)

Nonostante il nome, serve per **classificare** (es. Spam vs Non-Spam). Non possiamo usare una retta perché l'output deve essere una probabilità tra 0 e 1.

La Funzione Sigmoidale

Schiacciamo l'output lineare $z = w^T x$ attraverso una funzione non lineare:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- Se $z \rightarrow +\infty$, $\sigma(z) \rightarrow 1$.
- Se $z \rightarrow -\infty$, $\sigma(z) \rightarrow 0$.
- Se $z = 0$, $\sigma(z) = 0.5$ (Incertezza massima).

La Loss: Binary Cross-Entropy

Non usiamo l'MSE qui (non sarebbe convesso). Usiamo la **Log-Likelihood negativa**.

$$J(w) = - \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Spiegazione: Se il vero $y = 1$, vogliamo massimizzare $\log(\hat{y})$. Se $y = 0$, vogliamo massimizzare $\log(1 - \hat{y})$. Questa formula penalizza esponenzialmente le predizioni sbagliate "sicure" (es. predire 0.99 quando è 0).

1.4.5 4. Valutazione: Oltre l'Accuratezza

Questa sezione è critica. Dire "il mio modello ha il 90% di accuratezza" non significa nulla se il dataset è sbilanciato (es. 90% sani, 10% malati). Se predico sempre "sano", ho il 90% di accuratezza ma un modello inutile.

La Matrice di Confusione

	Predetto Positivo	Predetto Negativo
Reale Positivo	True Positive (TP)	False Negative (FN)
Reale Negativo	False Positive (FP)	True Negative (TN)

Le Metriche Chiave

- **Precision (Precisione):** $\frac{TP}{TP+FP}$. *"Di tutti quelli che ho chiamato 'Positivi', quanti lo erano davvero?"* Cruciale per i filtri anti-spam (non voglio buttare mail importanti).
- **Recall (Sensibilità):** $\frac{TP}{TP+FN}$. *"Di tutti i Positivi che esistevano al mondo, quanti ne ho trovati?"* Cruciale per diagnosi mediche (non voglio mandare a casa un malato dicendogli che è sano).
- **F1-Score:** La media armonica tra Precision e Recall. Un buon compromesso.

ROC Curve e AUC

Spesso il modello ci dà una probabilità (es. 0.7). Dobbiamo scegliere una soglia (Threshold) per decidere se è 0 o 1.

- **ROC Curve:** Disegna come cambiano TP Rate e FP Rate al variare della soglia.
- **AUC (Area Under Curve):** Un numero magico tra 0.5 (random) e 1.0 (perfetto). *Interpretazione Probabilistica (Importante!):* L'AUC è la probabilità che il modello dia un punteggio più alto a un positivo scelto a caso rispetto a un negativo scelto a caso. È una misura di "separabilità" delle classi, indipendente dalla soglia scelta.

1.5 Alberi di Decisione ed Ensemble Methods

Riferimento: *FDS07-Trees_Ensembles.pdf*

Finora abbiamo visto modelli (Lineari, Neural Networks) che assumono una forma geometrica fissa (es. una retta o un iperpiano). Ora cambiamo approccio: usiamo modelli che **tagliano lo spazio** in rettangoli, facendo una serie di domande sì/no.

1.5.1 1. Parametric vs Non-Parametric

- **Parametrici (es. Regressione Lineare):** Hanno un numero fisso di parametri (pesi w). La complessità del modello è bloccata, non importa quanti dati abbiamo.
- **Non-Parametrici (es. Alberi, k-NN):** La complessità del modello **cresce** con i dati. Più dati abbiamo, più l'albero diventa profondo e ramificato. Possono approssimare qualsiasi funzione.

1.5.2 2. Decision Trees (Alberi di Decisione)

Un albero è un diagramma di flusso. Per classificare un dato x , partiamo dalla radice e facciamo domande:

- "L'età è > 18 ?" $\rightarrow$ Sì $\rightarrow$ vai a destra.
- "Ha un lavoro?" $\rightarrow$ No $\rightarrow$ vai a sinistra.
- **Foglia:** Hai finito. Predici la classe di maggioranza in quella foglia.

Come si costruisce un albero? (Training)

L'albero viene costruito in modo **Greedy** (avido) e ricorsivo. A ogni nodo, cerchiamo la feature e la soglia che "puliscono" meglio i dati (separano meglio le classi). Per misurare la "purezza", usiamo due metriche:

A. Entropia (Shannon Entropy): Misura il disordine.

$$H(S) = - \sum p_i \log_2(p_i)$$

- Se ho 100% classe A: Entropia = 0 (Ordine perfetto).
- Se ho 50% A e 50% B: Entropia = 1 (Caos massimo).

Information Gain: È la differenza tra l'entropia prima del taglio e l'entropia media dopo il taglio. Scegliamo il taglio che massimizza il guadagno di informazione.

B. Gini Impurity: Simile all'entropia ma più veloce da calcolare (niente logaritmi).

$$Gini = 1 - \sum p_i^2$$

È la probabilità di sbagliare classificando un elemento a caso secondo la distribuzione delle classi nel nodo.

Il Problema dell'Overfitting

Gli alberi amano l'overfitting. Se non li fermiamo, continueranno a fare domande finché ogni foglia avrà un solo dato (Imparano a memoria il rumore). **Soluzioni (Pruning):**

- **Pre-Pruning:** Fermati se l'albero è profondo D livelli o se un nodo ha meno di N esempi.
- **Post-Pruning:** Costruisci l'albero intero, poi taglia i rami che non migliorano l'accuratezza su un validation set.

1.5.3 3. Ensemble Methods: L'Unione fa la Forza

Un singolo albero è instabile (cambi un dato, cambia tutto l'albero). L'idea degli Ensemble è combinare molti modelli "deboli" per ottenerne uno "forte".

A. Bagging (Bootstrap Aggregating)

Filosofia: Democrazia parallela.

1. Creiamo T dataset diversi campionando *con rimpiazzo* dal dataset originale (Bootstrap).
2. Addestriamo T alberi in parallelo, uno su ogni dataset.
3. **Voto:** Per predire, facciamo votare tutti gli alberi (media per regressione, maggioranza per classificazione).

Random Forest: È il re del Bagging. Aggiunge un trucco: a ogni taglio, l'albero può scegliere solo tra un **sottoinsieme casuale di feature**. Questo "decorrela" gli alberi (li costringe a essere diversi), rendendo la foresta molto più robusta.

B. Boosting

Filosofia: Miglioramento sequenziale. Invece di addestrare alberi in parallelo, li addestriamo **in serie**. Ogni nuovo albero cerca di correggere gli errori fatti da quelli precedenti.

AdaBoost (Adaptive Boosting):

- Assegna un peso a ogni dato. All'inizio tutti pesano uguale.
- Addestra un albero debole (spesso un "stump", un albero con un solo taglio).
- Aumenta il peso degli esempi sbagliati.
- Il prossimo albero si concentrerà sui dati difficili.

Gradient Boosting (GBM): È la versione generalizzata.

- Il primo albero F_0 predice la media.
- Calcoliamo i **Residui** (Errore = Vero Valore - Predizione).
- Il secondo albero h_1 cerca di predire i residui (cioè l'errore del primo).
- Aggiorniamo il modello: $F_1(x) = F_0(x) + \alpha h_1(x)$.

- Ripetiamo. Ogni albero "limita" l'errore del precedente.

XGBoost (eXtreme Gradient Boosting): È l'implementazione "dopata" del Gradient Boosting. Include regolarizzazione, parallelizzazione e gestione ottimizzata della memoria. È lo standard de-facto per dati tabulari.

1.5.4 4. Riassunto Finale

Algoritmo	Strategia	Obiettivo
Decision Tree	Divide et Impera	Interpretabilità, Semplicità
Random Forest	Bagging (Parallelo)	Ridurre la Varianza (Overfitting)
Gradient Boosting	Boosting (Sequenziale)	Ridurre il Bias (Underfitting)

Capitolo 2

Deep Learning

2.1 Dalle Feature Manuali alle Reti Neurali (Shallow Networks)

In questa sezione esploriamo il passaggio fondamentale dal Machine Learning classico al Deep Learning, analizzando i blocchi costruttivi di base: il neurone artificiale (Perceptron) e come combinarlo per risolvere problemi non lineari.

2.1.1 1. Il Cambio di Paradigma: Representation Learning

Nel Machine Learning tradizionale (es. SVM, Regressione Logistica), la performance dipende quasi interamente dalla qualità delle feature fornite in input.

- **Approccio Classico (Manual Features):** Un esperto di dominio deve costruire a mano le caratteristiche. Per classificare le email di Spam, un umano potrebbe creare regole come: *"Contiene la parola Viagra?"* oppure *"Ci sono più di 3 punti esclamativi?"*. *Problema:* È lento, costoso e fragile (se dimentichiamo una regola importante, il modello fallisce).
- **Representation Learning:** L'obiettivo è lasciare che sia il modello stesso a **imparare** le feature utili direttamente dai dati grezzi (raw data), senza intervento umano.
- **Deep Learning:** È una tecnica di Representation Learning che usa una gerarchia di strati per imparare concetti sempre più complessi (es. bordi → forme → oggetti).

2.1.2 2. Il Perceptron (Il Neurone Singolo)

Il Perceptron è l'unità base, ispirata al neurone biologico. Funziona in tre passi:

1. **Somma Pesata:** Moltiplica ogni input x_i per un peso w_i e aggiunge un bias b .

$$z = \sum (w_i x_i) + b = w^T x + b$$

2. **Attivazione (Step Function):** Applica una soglia rigida.

$$y = \begin{cases} 1 & \text{se } z > 0 \\ 0 & \text{altrove} \end{cases}$$

Intuizione: Geometrica del Perceptron

Geometricamente, un Perceptron definisce un **iperpiano** (una linea in 2D) che taglia lo spazio in due. Tutto ciò che sta da una parte è Classe 1, dall'altra è Classe 0.

2.1.3 3. Esercizio Svolto: Il Problema dello XOR

Domanda: Perché un singolo Perceptron non può imparare la funzione logica XOR?

Svolgimento: Analizziamo la tabella di verità dello XOR (OR Esclusivo):

x_1	x_2	y (XOR)
0	0	0
0	1	1
1	0	1
1	1	0

Se disegniamo questi punti su un piano cartesiano:

- I punti (0, 0) e (1, 1) hanno etichetta **0**.
- I punti (0, 1) e (1, 0) hanno etichetta **1**.

Un singolo Perceptron è un classificatore **lineare**: può separare le classi solo tracciando una singola linea retta. Provate a disegnare una singola linea retta che separi i punti (0, 1) e (1, 0) dagli altri due. **È impossibile**. I punti dello XOR non sono linearmente separabili.

Conclusione: Questo limite ha causato il primo "Inverno dell'IA". Per risolvere lo XOR, abbiamo bisogno di più linee, ovvero di più neuroni combinati insieme (Multilayer Perceptron).

2.1.4 4. Multilayer Perceptron (MLP) e Funzioni di Attivazione

Per superare i limiti del Perceptron, combiniamo più neuroni in strati successivi:

- **Input Layer:** I dati grezzi.
- **Hidden Layer(s):** Strati intermedi che trasformano i dati.
- **Output Layer:** La previsione finale.

Perché servono le Funzioni di Attivazione Non-Lineari?

Questa è una domanda d'esame fondamentale. Se usassimo solo somme pesate (funzioni lineari) in ogni strato, l'intera rete collasserebbe in un singolo strato lineare.

$$W_2(W_1x) = (W_2W_1)x = W_{tot}x$$

Per imparare funzioni complesse (come lo XOR o riconoscere immagini), dobbiamo introdurre delle **non-linearità** dopo ogni somma pesata.

Le funzioni principali sono:

1. **Sigmoide** ($\sigma(z) = \frac{1}{1+e^{-z}}$): Schiaccia l'output tra 0 e 1. *Pro*: Interpretabile come probabilità. *Contro*: Soffre del problema del "Vanishing Gradient" (derivata quasi zero agli estremi).
2. **Tanh** ($\tanh(z)$): Schiaccia tra -1 e 1. *Pro*: Centrata in zero (meglio per l'ottimizzazione rispetto alla sigmoide). *Contro*: Anche lei soffre di Vanishing Gradient.
3. **ReLU (Rectified Linear Unit, $\max(0, z)$)**: *Funzionamento*: Se $z < 0$ spegne il neurone (0), se $z > 0$ lascia passare il valore (z). *Pro*: Risolve il Vanishing Gradient (la derivata è 1 per $z > 0$), calcolo velocissimo. *Standard*: È l'attivazione di default per le reti moderne.

2.2 Forward e Backward Propagation: Il Motore del Learning

Questa è la parte più tecnica e importante del modulo. Qui capiamo come la rete "pensa" (Forward) e come "impara" (Backward).

2.2.1 1. Forward Propagation (Il flusso in avanti)

La Forward Propagation è il processo che va dall'input all'output per generare una predizione. Immaginiamo una rete con 1 strato nascosto:

1. **Input:** Matrice X (dimensioni $N \times D$).

2. **Hidden Layer:**

$$Z_1 = XW_1 + b_1$$

$$H = \text{Attivazione}(Z_1) \quad (\text{es. ReLU})$$

3. **Output Layer:**

$$Z_2 = HW_2 + b_2$$

$$\hat{Y} = \text{Softmax}(Z_2) \quad (\text{Probabilità finali})$$

2.2.2 2. La Loss Function (Come misuriamo l'errore)

Per addestrare la rete, dobbiamo quantificare quanto "sbaglia". Usiamo una funzione di costo $J(\theta)$.

- **Per la Regressione:** Usiamo spesso l'MSE (Errore Quadratico Medio).
- **Per la Classificazione:** Usiamo la **Cross-Entropy Loss** (o Log-Loss).

$$L = - \sum (y \cdot \log(\hat{y}))$$

Questa funzione penalizza pesantemente la rete se assegna una probabilità bassa alla classe corretta.

2.2.3 3. Backpropagation (L'Algoritmo di Apprendimento)

La Backpropagation è il metodo per calcolare il **gradiente** della Loss rispetto a tutti i pesi della rete (∇W). Senza questo algoritmo, non potremmo dire alla rete "come" modificare i pesi per ridurre l'errore.

Intuizione: La Chain Rule (Regola della Catena)

La Backprop si basa interamente sulla Chain Rule del calcolo differenziale. Se hai una funzione composta $y = f(g(x))$, la derivata di y rispetto a x è:

$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial g} \cdot \frac{\partial g}{\partial x}$$

In una rete neurale, l'errore è la "tessera finale del domino". Vogliamo sapere quale spinta dare alla prima tessera (i pesi) per far cadere l'ultima nel modo giusto. Moltiplichiamo le derivate tornando indietro dalla fine all'inizio.

Esercizio Svolto: Derivazione dei Gradienti (Slide 45-55)

Obiettivo: Calcolare le formule di aggiornamento per i pesi W_1 e W_2 in una rete Shallow.

Il Modello: 1. Input X 2. Hidden: $H = \text{ReLU}(XW_1)$ 3. Output: $Z = HW_2 + b$ 4. Predizione: $\hat{Y} = \sigma(Z)$ (Sigmoid) 5. Loss: Binary Cross Entropy

[cite_start]**Passo 1: Il Gradiente dell'Errore Finale** (δ_{out})[cite : 55]*Vogliamo la derivata della Loss rispetto alla Entropy, otteniamo un risultato pulito: $\frac{\partial \mathcal{L}}{\partial Z} = (\hat{Y} - Y)$* Questo è il nostro "segnale di errore" iniziale. Se la predizione è alta e il vero target è basso (o viceversa), il gradiente è grande.

[cite_start]**Passo 2: Gradiente per i Pesi di Output** (W_2)[cite : 55]*Collegiamo l'errore ai pesi W_2 .* La derivata dipende dall'attivazione che precede i pesi (H).

$$\frac{\partial \mathcal{L}}{\partial W_2} = H^T \cdot (\hat{Y} - Y)$$

[cite_start]**Passo 3: Propagare l'errore allo Strato Nascosto**[cite : 55]*Or dobbiamo portare l'errore indietro per calcolare l'errore sui neuroni nascosti.*

$$\delta_{hidden} = (\hat{Y} - Y) \cdot W_2^T$$

[cite_start]**Passo 4: Attraversare la ReLU**[cite : 55]*Dobbiamo derivare la funzione di attivazione ReLU* ≤ 0 . Quindi, "spegniamo" il gradiente per i neuroni che non si erano attivati.

$$\delta_{relu} = \delta_{hidden} \odot \mathbb{I}(Z_1 > 0)$$

Dove $\odot$ è il prodotto elemento per elemento.

[cite_start]**Passo 5: Gradiente per i Pesi di Input** (W_1)[cite : 55]*In fine, colleghiamo questo errore* $X^T \cdot \delta_{relu}$

Intuizione: Interpretazione Finale

Le formule sembrano complesse, ma dicono questo: "Aggiorna il peso w_{ij} in proporzione all'errore commesso, ma **solo se** quel neurone era attivo (grazie alla ReLU) e in proporzione a quanto era forte l'input X che lo ha stimolato."

2.3 Deep Neural Networks: Architetture e Ottimizzazione Avanzata

In questa sezione abbandoniamo le reti "shallow" (poco profonde) per entrare nel mondo del **Deep Learning**. Analizzeremo perché la profondità è fondamentale, come si addestrano reti complesse e quali tecnologie moderne (ResNet, Adam, Batch Norm) hanno reso possibile la rivoluzione dell'IA.

2.3.1 1. L'Intuizione Geometrica: La "Piegatura" dello Spazio

Perché aggiungere più strati invece di allargare un singolo strato? Le slide introducono l'analogia del "*Folding*" (Piegatura).

Intuizione: L'Analogia del Foglio di Carta

Immagina i tuoi dati come punti disegnati su un foglio di carta. Se i dati sono complessi (es. una spirale rossa e blu), non puoi separarli con un singolo taglio netto (una retta).

- **Layer 1 (Folding):** Il primo strato della rete non cerca di classificare subito. Invece, "piega" il foglio su se stesso. Punti che erano lontani nello spazio originale possono finire sovrapposti o allineati.
- **Layer 2 (Cutting):** Il secondo strato opera su questo spazio piegato. Ora, grazie alla piegatura precedente, una semplice retta può separare le classi che prima erano intrecciate.

Conclusion: Ogni strato profondo crea una rappresentazione sempre più astratta e "facile" da gestire per lo strato successivo.

2.3.2 2. Formulazione Matematica di una Rete Profonda

Matematicamente, una rete profonda è una composizione ricorsiva di funzioni non lineari. Se definiamo $a[\cdot]$ come la funzione di attivazione, l'output y di una rete a K strati è:

$$y = \beta_K + \Omega_K \cdot a[\beta_{K-1} + \Omega_{K-1} \cdot a[\dots a[\beta_0 + \Omega_0 x] \dots]]$$

Dove:

- Ω_l : Matrice dei pesi del livello l .
- β_l : Vettore dei bias del livello l .
- h_l : Output dello strato nascosto l , calcolato come $h_l = a[\beta_{l-1} + \Omega_{l-1}h_{l-1}]$.

2.3.3 3. Le Sfide dell'Ottimizzazione

Addestrare reti profonde è esponenzialmente più difficile che addestrare reti shallow. I nemici principali sono:

1. **Paesaggio Non-Convesso:** La Loss Function non è una semplice ciotola. È piena di *Minimi Locali* (buche poco profonde) e *Punti di Sella* (zone piatte dove il gradiente si ferma, ma non siamo al minimo).

2. **Narrow Valleys (Valli Strette):** Il percorso verso il minimo è spesso una gola stretta e ripida. Il Gradient Descent classico rimbalza tra le pareti della valle invece di scendere lungo il fondo.
3. **Vanishing/Exploding Gradient:** Il gradiente tende a sparire (diventare zero) o esplodere (diventare infinito) attraversando molti strati.

2.3.4 4. Evoluzione degli Algoritmi di Ottimizzazione

Per superare questi ostacoli, sono stati inventati algoritmi più intelligenti del semplice SGD.

A. Momentum (L’Inerzia)

Problema: SGD oscilla troppo nelle valli strette e si blocca nei punti di sella. **Soluzione:** Aggiungiamo un termine di "inerzia" (momentum), come una pallina pesante che rotola.

$$v_t = \mu v_{t-1} - \alpha \nabla L(\theta)$$

$$\theta_t = \theta_{t-1} + v_t$$

La velocità v_t accumula i gradienti passati. Se andiamo sempre nella stessa direzione, acceleriamo. Se il gradiente cambia direzione (oscillazione), l’inerzia ci stabilizza.

B. Nesterov Momentum (Guardare Avanti)

Miglioramento: Invece di calcolare il gradiente dove siamo *ora*, calcoliamolo nel punto *dove saremo dopo il passo di momentum*. È come un pilota che guarda la curva successiva prima di sterzare.

$$v_t = \mu v_{t-1} - \alpha \nabla L(\theta_{t-1} + \mu v_{t-1})$$

C. Adaptive Learning Rates (Adattarsi ai Parametri)

Non tutti i parametri della rete imparano alla stessa velocità.

- **AdaGrad:** Rallenta il learning rate per i parametri che hanno ricevuto gradienti molto alti (frequenti) e lo accelera per quelli rari. *Difetto:* Il learning rate decresce troppo in fretta e si ferma.
- **RMSProp:** Corregge AdaGrad usando una media mobile esponenziale dei quadrati dei gradienti. Permette di continuare ad apprendere indefinitamente.

D. Adam (Adaptive Moment Estimation)

Lo Standard Moderno: Adam combina il meglio di Momentum e RMSProp. Tieni traccia di due cose:

1. Media dei gradienti (Momentum, β_1).
2. Media dei gradienti al quadrato (RMSProp, β_2).

È robusto, veloce e richiede poco tuning degli iperparametri (default tipico: $\alpha = 0.001$).

2.3.5 5. Il Problema del Vanishing Gradient e le Soluzioni

In una rete profonda, la Backpropagation calcola il gradiente usando la Chain Rule (moltiplicazioni successive).

$$\text{Gradiente Totale} = \frac{\partial L}{\partial y} \cdot \dots \cdot \sigma'(z_2) \cdot w_2 \cdot \sigma'(z_1) \cdot w_1$$

Se usiamo la Sigmoidale (σ), la sua derivata massima è 0.25. Dopo 10 strati, il gradiente diventa $0.25^{10} \approx 0.000001$. La rete smette di imparare nei primi strati.

Soluzione 1: Inizializzazione dei Pesi

Non possiamo inizializzare i pesi a zero (simmetria) o a caso (instabilità).

- **Xavier (Glorot) Initialization:** Ottimale per Sigmoidale/Tanh. Mantiene la varianza dell'attivazione costante tra i layer.

$$W \sim \mathcal{N}\left(0, \frac{1}{D_{in} + D_{out}}\right)$$

- **Kaiming (He) Initialization:** Ottimale per ReLU. Tiene conto che la ReLU spegne metà dei neuroni.

$$W \sim \mathcal{N}\left(0, \frac{2}{D_{in}}\right)$$

Soluzione 2: Funzione di Attivazione ReLU

La **ReLU** (Rectified Linear Unit) è stata la chiave di volta.

$$f(x) = \max(0, x)$$

La sua derivata è **1** per $x > 0$. Moltiplicare per 1 all'infinito non riduce il gradiente! Il segnale di errore fluisce intatto fino all'inizio della rete.

2.3.6 6. Residual Networks (ResNet)

Anche con la ReLU, reti molto profonde (100+ strati) erano difficili da addestrare. **Idea:** Invece di imparare la funzione completa $H(x)$, impariamo solo il residuo (la differenza) $F(x) = H(x) - x$. L'output del blocco diventa:

$$H(x) = F(x) + x$$

Quel $+x$ è una **Skip Connection** (o scorciatoia).

Intuizione: L'Autostrada del Gradiente

Durante la Backpropagation, la derivata di $(F(x) + x)$ è $(F'(x) + 1)$. Il termine **+1** garantisce che ci sia sempre un canale aperto attraverso cui il gradiente può scorrere all'indietro senza essere moltiplicato per pesi piccoli. Questo permette di addestrare reti con migliaia di strati.

2.3.7 7. Batch Normalization (BN)

Le reti faticano se la distribuzione dei dati in ingresso cambia continuamente ("Internal Covariate Shift"). La BN normalizza l'input di ogni strato per avere media 0 e varianza 1, usando le statistiche del mini-batch corrente.

$$\hat{x} = \frac{x - \mu_{batch}}{\sqrt{\sigma_{batch}^2 + \epsilon}}$$

Poi applica una scala e traslazione apprendibile ($\gamma\hat{x} + \beta$) per non limitare la capacità della rete. **Attenzione:**

- **Training:** Usa media/varianza del batch corrente.
- **Test:** Usa una media mobile (running average) calcolata durante tutto il training (poiché in test potremmo avere un solo campione).

2.3.8 8. Regularizzazione: Dropout

Il Dropout è una tecnica brutale ma efficace per prevenire l'Overfitting. Durante il training, **spegliamo** (mettiamo a zero) ogni neurone con probabilità p (es. 0.5).

- **Perché funziona?** Impedisce ai neuroni di "co-adattarsi" troppo. Nessun neurone può fare affidamento sulla presenza di un altro specifico neurone, quindi deve imparare feature più robuste e indipendenti. È come allenare un ensemble di milioni di sottoreti diverse.
- **Inverted Dropout:** Per mantenere coerente la magnitudine dell'output, dividiamo i neuroni attivi per $(1 - p)$ durante il training. In questo modo, durante il test (dove tutti i neuroni sono attivi), non dobbiamo fare nulla.

2.4 Self-Supervised Learning e Masked Modeling

Fino ad ora abbiamo parlato di *Supervised Learning*, dove per ogni immagine abbiamo un'etichetta (es. "Gatto"). Ma le etichette costano care (umani devono annotarle). I dati non etichettati (immagini su internet) sono invece infiniti e gratuiti. Il **Self-Supervised Learning (SSL)** è la tecnica per imparare rappresentazioni robuste dai dati *senza* etichette umane, creando un "problema finto" (Pretext Task) che la rete deve risolvere usando solo i dati stessi.

2.4.1 Masked Signal Modeling

L'idea dominante oggi deriva dal NLP (BERT) ed è stata portata nella Visione: il **Masking**. Il concetto è semplice ma potente:

1. Prendiamo un dato x (es. una frase o un'immagine).
2. Corrompiamo il dato nascondendo una parte di esso (Masking) $\rightarrow \tilde{x}$.
3. Chiediamo alla rete neurale di ricostruire la parte mancante originale x partendo da $\tilde{x}$.

Per fare questo, la rete deve "capire" il contesto e la struttura dei dati. Se nascondo metà faccia di un cane, la rete deve aver imparato come è fatto un cane per ridisegnare l'occhio mancante.

2.4.2 Masked Autoencoder (MAE)

Il **Masked Autoencoder** (He et al., 2021) è l'architettura stato dell'arte per il self-supervised learning nelle immagini. Si basa sui Vision Transformers (ViT) ma introduce un design **asimmetrico** fondamentale per l'efficienza.

1. Architettura Asimmetrica

A differenza degli Autoencoder classici (dove Encoder e Decoder sono speculari), il MAE ha:

- **Encoder (Grande e Potente):** Vede *solo* le patch visibili (non mascherate).
- **Decoder (Piccolo e Leggero):** Deve ricostruire i pixel mancanti usando le feature latenti prodotte dall'encoder e dei "Mask Tokens" (segnaposto).

2. Il Processo di Training

Ecco come funziona passo-passo un MAE:

1. **Patching:** L'immagine viene divisa in patch (es. 16×16 pixel).
2. **Masking Estremo:** Mascheriamo una percentuale altissima di patch (tipicamente il **75%**). *Perché così tanto?* Le immagini sono segnali molto ridondanti (un pixel è quasi uguale al suo vicino). Se mascherassimo solo il 15% (come in NLP), la rete copierebbe solo i vicini senza imparare la semantica globale. Togliendo il 75%, costringiamo la rete a ragionare ad alto livello ("Qui c'è una ruota, quindi sopra deve esserci un parafrangente").

3. ****Encoding:**** Solo il 25% delle patch (quelle visibili) entra nell'Encoder. Questo rende il training velocissimo (costo quadratico ridotto drasticamente).
4. ****Decoding:**** Le patch codificate vengono unite ai Mask Tokens (che dicono "qui manca qualcosa") e passate al Decoder leggero per ricostruire l'immagine intera.
5. ****Loss Function:**** Calcoliamo l'errore (MSE) tra l'immagine originale e quella ricostruita, ma **solo sui pixel mascherati**. Non ci interessa se la rete ricostruisce bene ciò che ha già visto.

3. Fine-Tuning

Una volta pre-addestrato il MAE su milioni di immagini senza etichette, buttiamo via il Decoder. Usiamo l'Encoder (che ha imparato a estrarre feature visive potentissime) come base per un classificatore supervisionato, ottenendo prestazioni superiori rispetto al training da zero.

2.5 Image Filtering: I Fondamenti della Visione Artificiale

Riferimento: *FDS10-Image Filtering.pdf*

Prima di dare in pasto le immagini alle reti neurali, dobbiamo capire cosa sono le immagini per un computer e come elaborarle matematicamente.

2.5.1 1. L'Immagine come Funzione

Un'immagine non è altro che una funzione discreta $f(x, y)$ che mappa una posizione spaziale (x, y) a un valore di intensità (pixel).

- **Grayscale:** $f(x, y) \in [0, 255]$ (un singolo canale).
- **RGB (Colori):** $f(x, y) = [r(x, y), g(x, y), b(x, y)]^T$. È un vettore di 3 valori.

Intuizione: Combinatoria dei Pixel

Quante immagini diverse di dimensione $N \times N$ esistono? Se ogni pixel ha 256 valori possibili, le combinazioni sono $256^{(N \times N)}$. Un numero astronomico! Questo ci dice che lo spazio delle immagini è enorme e sparso: la maggior parte delle combinazioni casuali di pixel è solo rumore statico ("effetto neve").

2.5.2 2. Il Filtraggio (Smoothing)

Spesso le immagini reali contengono **rumore**. Il nostro primo obiettivo è pulirle. L'idea base è: il valore "vero" di un pixel dovrebbe essere simile a quello dei suoi vicini.

Moving Average (Box Filter)

Sostituiamo ogni pixel con la media dei suoi vicini in una finestra $k \times k$.

$$\text{Output}[i, j] = \frac{1}{k^2} \sum_{u=-k/2}^{k/2} \sum_{v=-k/2}^{k/2} \text{Input}[i + u, j + v]$$

Problema: Questo filtro crea artefatti quadrati ("blocky artifacts") e non è naturale. Tratta il vicino immediato allo stesso modo del vicino lontano.

Il Filtro Gaussiano

È la soluzione elegante. Invece di una media semplice, usiamo una media ponderata dove i pesi decadono con la distanza dal centro, seguendo una campana di Gauss. Formula 2D:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

- **Parametro σ (Sigma):** Controlla la "sfocatura". Un σ grande sfoca di più (i pesi sono distribuiti su un'area più ampia).
- **Simmetria Rotazionale:** Il filtro è circolare, non privilegia direzioni orizzontali o verticali.

Teoria: Proprietà Fondamentale: La Separabilità

Questa è una domanda da esame: "**Perché il filtro Gaussiano è efficiente?**"
La risposta è la **Separabilità**. L'esponenziale 2D si può spezzare nel prodotto di due esponenziali 1D:

$$e^{-(x^2+y^2)} = e^{-x^2} \cdot e^{-y^2}$$

Conseguenza pratica: Invece di fare una convoluzione 2D (costosa, $O(K^2)$ operazioni per pixel), possiamo fare: 1. Una convoluzione 1D sulle righe (asse x). 2. Una convoluzione 1D sulle colonne (asse y) del risultato. Il costo scende da K^2 a $2K$. Per filtri grandi, il risparmio è enorme.

2.6 Architectures for Grids: Convolutional Neural Networks (CNN)

Riferimento: FDS10_bis-Grid Architecture 2025.pdf

Qui avviene il salto di qualità: passiamo dal processare immagini "a mano" (filtri fissi) a imparare i filtri stessi (Deep Learning).

2.6.1 1. Perché le Reti Fully Connected (MLP) falliscono sulle immagini?

Se proviamo a usare un MLP classico su un'immagine, incontriamo due problemi fatali:

1. **Esplosione dei Parametri:** Un'immagine modesta $200 \times 200 \times 3$ ha 120.000 input. Se il primo strato nascosto ha 1000 neuroni, la matrice dei pesi è $120.000 \times 1000 = 120$ Milioni di pesi. Impossibile da addestrare (overfitting immediato).
2. **Perdita della Struttura Spaziale:** L'MLP "appiattisce" (flatten) l'immagine in un vettore. Non sa che il pixel (0,0) è vicino al pixel (0,1). Perde la nozione di "vicinanza" e "forma".

2.6.2 2. La Soluzione: Convolutional Neural Networks (CNN)

Le CNN introducono un forte **Inductive Bias** (ipotesi a priori sul mondo):

- **Località:** Un neurone deve guardare solo una piccola finestra locale dell'immagine (Receptive Field).
- **Translation Equivariance:** Se un filtro è utile per trovare un occhio in alto a sinistra, lo stesso filtro è utile per trovarlo in basso a destra. **Condividiamo i pesi (Weight Sharing)** su tutta l'immagine.

2.6.3 3. Anatomia di un Layer Convoluzionale

Questo è il blocco costruttivo fondamentale. Devi conoscere le dimensioni a memoria.

Input: Tensore ($H \times W \times C_{in}$) (Altezza, Larghezza, Canali Input). **Filtro (Kernel):** Tensore ($K \times K \times C_{in}$). **Attenzione:** Il filtro ha la stessa profondità C_{in} dell'input! **Operazione:** Il filtro scorre spazialmente (su H e W). In ogni posizione, fa il prodotto scalare attraverso **tutti** i canali di input e somma tutto in un unico numero.

Intuizione: Da dove escono i Canali di Output?

Un singolo filtro produce una singola mappa 2D (Feature Map). Se vogliamo C_{out} feature maps in uscita (es. 64), dobbiamo usare **64 filtri diversi**.

Dimensioni Pesi Totali: $C_{out} \times (K \times K \times C_{in})$.

2.6.4 4. Iperparametri Chiave: Stride, Padding, Dilation

Oltre alla dimensione del kernel K , questi parametri controllano la geometria dell'output.

- **Stride (S):** Il passo di scorrimento.

- $S = 1$: Scorre su ogni pixel. Output grande.
- $S = 2$: Salta un pixel. L'output si dimezza (Downsampling).
- **Padding (P):** Cornice di zeri aggiunta attorno all'input.
 - *Valid Padding*: Nessun padding. L'immagine si rimpicciolisce a ogni strato.
 - *Same Padding*: Padding calcolato affinché Output Size = Input Size. Fondamentale nelle reti profonde (es. ResNet) per sommare tensori.
- **Dilation:** "Gonfia" il kernel inserendo buchi tra i pesi. Permette di avere un campo ricettivo enorme senza aumentare i parametri. Usato molto nella segmentazione semantica.

Formula Dimensione Output:

$$W_{out} = \left\lfloor \frac{W_{in} - K + 2P}{S} \right\rfloor + 1$$

2.6.5 5. Pooling: Invarianza e Compressione

Dopo la convoluzione (e la ReLU), spesso applichiamo il **Pooling**.

- **Max Pooling:** Prende il valore massimo in una finestra (es. 2×2).
- **Average Pooling:** Prende la media.

Scopo: 1. Ridurre le dimensioni spaziali (meno calcoli dopo). 2. Rendere la rete **invariante** a piccole traslazioni (se spostato l'occhio di 1 pixel, il max nella finestra 2×2 è probabilmente lo stesso).

2.6.6 6. Architetture Famose (Evoluzione Storica)

1. **AlexNet (2012):** La prima CNN profonda. Filtri grandi (11×11), stride aggressivi. Ha dimostrato che il Deep Learning funziona.
2. **VGG (2014):** Filosofia della semplicità. Usa **solo filtri** 3×3 ovunque. *Perché?* Due strati 3×3 hanno lo stesso campo ricettivo di un 5×5 , ma con meno parametri e più non-linearità in mezzo. "Deep and Narrow".
3. **ResNet (2015):** Introduce le *Skip Connections* (viste nel capitolo precedente) per addestrare reti profondissime (100+ layer).

2.6.7 7. Receptive Field (Campo Ricettivo)

È la porzione dell'immagine di input che influenza un singolo pixel dell'output. In una rete profonda, anche se i filtri sono piccoli (3×3), il campo ricettivo cresce linearmente con la profondità. Un pixel nell'ultimo strato di una ResNet "vede" quasi tutta l'immagine originale. Questo permette alla rete di capire il contesto globale (es. "questo pixel grigio è orecchio di elefante o sasso? Dipende se vedo una proboscide vicino").

2.7 Memory and Sequence Modeling: Gestire il Tempo

Riferimento: *FDS11_bis-Mem Seq 2025.pdf*

Fino ad ora abbiamo trattato i dati come entità statiche (immagini). Ma il mondo è dinamico: il linguaggio, i video, le serie storiche finanziarie sono **sequenze**. In questo capitolo introduciamo l'architettura regina per gestire il tempo: le **Recurrent Neural Networks (RNN)**.

2.7.1 1. Perché il Tempo è Speciale?

Non possiamo trattare il tempo semplicemente come una dimensione spaziale in più (come aggiungiamo la profondità in un volume 3D). Il tempo ha proprietà uniche:

1. **Causalità (The Arrow of Time):** Il futuro non può influenzare il passato. Mentre in un'immagine un pixel a destra può influenzare la comprensione di un pixel a sinistra, nel tempo l'informazione scorre in una sola direzione.
2. **Memoria:** Per capire il presente, devo avere una rappresentazione compressa ("summary") di ciò che è successo prima.

Intuizione: L'Esempio del Gatto Frank

Nelle slide vediamo l'esempio del gatto "Frank". Se vedo solo la coda di un gatto (frame t), come faccio a sapere che è Frank? Devo aver visto la sua faccia nel frame $t - 5$ e aver "mantenuto in memoria" quell'informazione fino ad oggi. Senza memoria, ogni istante è isolato e incomprensibile.

2.7.2 2. Primo Approccio: Convoluzioni Causali (1D CNN)

Possiamo usare le CNN sulle sequenze? Sì, ma con una regola ferrea: **Causalità**. Se usiamo un filtro standard centrato sul tempo t , esso guarderebbe anche a $t + 1$ (il futuro). Questo è vietato in molti task (es. predire il mercato azionario).

- **Causal Convolution:** È una convoluzione standard, ma "shiftata" (spostata) in modo che al tempo t il filtro veda solo $[t, t - 1, t - 2 \dots]$.
- **Limite:** Per ricordare cose molto lontane nel passato, avremmo bisogno di filtri enormi o di reti profondissime. La CNN ha una memoria limitata dalla sua *Window Size*.

2.7.3 3. La Soluzione Elegante: Recurrent Neural Networks (RNN)

Le RNN risolvono il problema della memoria introducendo un **ciclo di feedback**. L'idea è processare un elemento alla volta, mantenendo uno "Stato Nascosto" (h_t) che funge da memoria.

L'Equazione Fondamentale della RNN

Al tempo t , il neurone riceve due input: 1. Il dato corrente (x_t). 2. Il suo stesso stato al passo precedente (h_{t-1}).

$$h_t = \sigma(W_{hh}h_{t-1} + W_{xh}x_t + b)$$

$$y_t = g(W_{hy}h_t + c)$$

Dove:

- W_{hh} : Pesi che trasformano la memoria precedente (Hidden-to-Hidden).
- W_{xh} : Pesi che processano il nuovo input (Input-to-Hidden).
- W_{hy} : Pesi per calcolare l'output (Hidden-to-Output).

Teoria: Weight Sharing nel Tempo

La caratteristica più importante delle RNN è che le matrici W_{hh}, W_{xh}, W_{hy} sono **condivise (identiche)** per ogni passo temporale. La RNN applica la *stessa funzione* ripetutamente. Questo permette di gestire sequenze di lunghezza infinita con un numero fisso di parametri.

2.7.4 4. Deep RNN e Unrolling

Possiamo rendere le RNN "profonde" (Deep RNN) impilando più strati ricorrenti uno sopra l'altro. L'output del primo strato RNN diventa l'input del secondo strato RNN.

Concetto di "Unrolling" (Srotolamento): Per capire come funziona, immaginiamo di srotolare la rete nel tempo. Una RNN che gira per 5 secondi equivale a una rete Feed-Forward profonda 5 strati, dove ogni strato ha gli stessi pesi.

2.7.5 5. Backpropagation Through Time (BPTT)

Come addestriamo una rete con cicli? Srotolandola! 1. Eseguiamo la rete in avanti per T passi (Forward Pass) calcolando la Loss totale (somma delle Loss a ogni istante).

$$J = \sum_{t=1}^T \mathcal{L}(y_t, \text{target}_t)$$

2. Calcoliamo il gradiente tornando indietro dal tempo T al tempo 0.

Il Problema del Gradiente: Poiché moltiplichiamo per la stessa matrice W_{hh} ripetutamente (una volta per ogni step temporale):

- Se gli autovalori di $W_{hh} < 1$: Il gradiente tende a zero esponenzialmente (**Vanishing Gradient**). La rete dimentica il passato lontano.
- Se gli autovalori di $W_{hh} > 1$: Il gradiente esplode (**Exploding Gradient**).

Questo rende difficile per le RNN standard imparare dipendenze a lungo termine (Long-Range Dependencies).

2.7.6 6. Sequence Modeling e NLP (Natural Language Processing)

Le RNN sono state la base per la modellazione del linguaggio. **Obiettivo:** Predire la parola successiva data la storia.

$$P(\text{"the cat sat on"}) = P(\text{the}) \cdot P(\text{cat}|\text{the}) \cdot P(\text{sat}|\text{the cat}) \dots$$

Rappresentazione delle Parole (One-Hot)

Come diamo in pasto le parole a una rete? Usiamo vettori **One-Hot**: un vettore lungo quanto il vocabolario (es. 10.000) con tutti zeri e un solo '1' alla posizione della parola corrente.

$$\mathbf{a} = [1, 0, 0, \dots], \quad \mathbf{aardvark} = [0, 1, 0, \dots]$$

Training vs Testing: Teacher Forcing

C'è una differenza cruciale tra come addestriamo e come usiamo una RNN generativa.

- **Inferenza (Testing):** La rete predice una parola $\hat{y}_t$. Questa parola predetta viene usata come *input* per il passo successivo x_{t+1} . Se la rete sbaglia, l'errore si propaga ("compounding error").
- **Training (Teacher Forcing):** Durante l'addestramento, noi conosciamo la frase vera ("Ground Truth"). Al tempo $t+1$, diamo in input alla rete la parola **vera**, non quella che lei ha predetto al passo prima. *Metafora:* È come un insegnante che ti corregge subito dopo ogni parola sbagliata invece di lasciarti finire la frase dicendo sciocchezze. Questo stabilizza e velocizza l'apprendimento.

2.7.7 7. Confronto Finale: Chi gestisce meglio la Memoria?

Le slide concludono confrontando le architetture sulla gestione della memoria passata:

- **CNN (Autoregressive):** Memoria limitata dalla dimensione del filtro (Receptive Field). Non vede oltre la finestra.
- **RNN:** Memoria teoricamente infinita (lo stato h_t si aggiorna sempre), ma praticamente limitata dal Vanishing Gradient. La memoria è compressa in un singolo vettore h_t (Collo di bottiglia).
- **Attention / Transformers (Hint):** Accesso diretto a qualsiasi punto del passato, senza compressione. (Sarà l'argomento delle prossime lezioni).

2.8 Transformers: La Rivoluzione dell'Attenzione

Riferimento: *FDS12-Transformers.pdf*

Se le CNN hanno risolto la visione e le RNN hanno provato a risolvere il tempo, i **Transformers** hanno unificato tutto sotto un unico meccanismo: l'**Attenzione**. Questo capitolo spiega l'architettura introdotta nel paper storico "*Attention is All You Need*" (2017), che ha eliminato la necessità di ricorrenze e convoluzioni.

2.8.1 1. I Limiti dei Predecessori

Prima di capire la soluzione, dobbiamo capire il problema.

1. **CNN (Località):** Le CNN sono bravissime a trovare pattern locali (bordi, occhi), ma faticano a collegare informazioni distanti nello spazio. Per collegare un pixel in alto a sinistra con uno in basso a destra, servono molti strati.
2. **RNN (Sequenzialità):** Le RNN processano una parola alla volta.
 - *Lentezza:* Non sono parallelizzabili (devi aspettare il tempo $t - 1$ per calcolare t).
 - *Memoria:* Faticano a ricordare informazioni viste 1000 passi fa (vanishing gradient), nonostante trucchi come le LSTM.

2.8.2 2. Innovazione 1: I Token (L'atomo del dato)

Il Transformer non lavora su stringhe di testo o immagini grezze. Lavora su **Token**.

Intuizione: Cos'è un Token?

Un token è l'unità minima di informazione semantica.

- **Nel Testo:** Può essere una parola ("mela") o una parte di parola ("ing" in "playing"). Ogni token viene convertito in un vettore numerico (Embedding) di dimensione fissa d (es. 512).
- **Nelle Immagini (Vision Transformer - ViT):** Un'immagine viene tagliata in una griglia di *patches* (es. quadratini 16×16). Ogni quadratino viene "appiattito" in un vettore e trattato esattamente come se fosse una parola in una frase.

2.8.3 3. Innovazione 2: L'Attenzione (Self-Attention)

Questa è la chiave di volta. Invece di guardare i vicini (CNN) o il passato immediato (RNN), l'Attenzione permette a ogni token di guardare **tutti gli altri token della sequenza contemporaneamente** e decidere quali sono importanti.

La Metafora del Database: Query, Key, Value

Per ogni token in ingresso, la rete apprende tre vettori distinti tramite matrici di pesi W_Q, W_K, W_V :

1. **Query (Q):** "Cosa sto cercando?". È la domanda che il token corrente pone agli altri.

2. **Key (K):** "Chi sono io?". È l'etichetta che ogni token espone per farsi trovare.
3. **Value (V):** "Che contenuto porto?". È l'informazione vera e propria che verrà estratta se c'è un match.

Analisi Grammaticale Frase: "*L'animale non ha attraversato la strada perché era troppo stanco.*" Quando processiamo la parola "**era**" (o il pronome sottinteso), la sua **Query** cerca il soggetto.

- La **Key** di "strada" dice "sono un oggetto inanimato". Match basso.
- La **Key** di "animale" dice "sono un essere vivente maschio". Match alto con "stanco".
- Risultato: L'attenzione si focalizza su "animale" e ne preleva il **Value**.

La Formula Matematica (Scaled Dot-Product Attention)

Questa formula è sacra. Devi saperla spiegare.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Analisi passo-passo:

1. **QK^T :** Facciamo il prodotto scalare tra la Query corrente e le Key di tutti gli altri. Il prodotto scalare misura la **similarità**. Se due vettori sono allineati, il valore è alto.
2. **$\sqrt{d_k}$ (Scaling):** Dividiamo per la radice della dimensione dei vettori. *Perché?* Se i vettori sono lunghi, il prodotto scalare può diventare enorme. Questo spingerebbe la Softmax nelle regioni piatte (gradiente zero). Lo scaling mantiene i gradienti stabili.
3. **softmax:** Trasforma i punteggi in probabilità (tutti positivi, somma 1). Questo decide "quanta attenzione" dare a ciascun token (es. 0.9 ad "animale", 0.1 a "strada").
4. **$\cdot V$:** Moltiplichiamo le probabilità per i vettori Value. Creiamo una somma pesata dei concetti rilevanti.

Multi-Head Attention

Un solo meccanismo di attenzione non basta. Una parola può avere più relazioni (soggetto, oggetto, aggettivo). La **Multi-Head Attention** esegue h meccanismi di attenzione **in parallelo**, ognuno con i propri pesi W_Q, W_K, W_V .

- La "Testa 1" potrebbe focalizzarsi sulla grammatica (soggetto-verbo).
- La "Testa 2" potrebbe focalizzarsi sul contesto semantico.
- Alla fine, i risultati vengono concatenati e proiettati linearmente.

2.8.4 4. Innovazione 3: Positional Encoding

Il meccanismo di Attenzione è, per natura, un insieme non ordinato (Set).

$\text{Attention}(\text{"Mario mangia la mela"}) == \text{Attention}(\text{"Mela la mangia Mario"})$

Poiché non ci sono ricorrenze o convoluzioni, il modello non sa l'ordine delle parole.

Soluzione: Iniettiamo l'ordine sommando ai vettori di embedding dei vettori di posizione fissi.

$$\text{Input} = \text{Embedding}(\textit{Token}) + \text{PositionalEncoding}(\textit{Pos})$$

Solitamente si usano funzioni **seno** e **coseno** di frequenze diverse. Questo permette al modello di imparare concetti come "vicino", "lontano" o "posizione relativa".

2.8.5 5. L'Architettura Completa (Encoder-Decoder)

Il Transformer originale (per la traduzione) ha due blocchi:

A. Encoder (Comprensione): Prende la frase di input, applica Multi-Head Attention e reti Feed-Forward. Produce una rappresentazione ricca e contestualizzata di ogni parola. *Esempio moderno:* **BERT** è un Encoder-only.

B. Decoder (Generazione): Genera la frase di output parola per parola (Autoregressivo). Ha due tipi di attenzione: 1. *Masked Self-Attention:* Guarda solo le parole già generate (non può sbirciare il futuro). 2. *Cross-Attention:* Usa le sue Query per interrogare le Key/Value prodotte dall'**Encoder**. È qui che avviene la traduzione ("Come si dice questa cosa che ha capito l'Encoder?"). *Esempio moderno:* **GPT** è un Decoder-only.

2.8.6 6. Vision Transformers (ViT)

Le slide mostrano che possiamo applicare questo anche alle immagini. Invece di usare CNN, dividiamo l'immagine in patch 16×16 , le appiattiamo e le proiettiamo linearmente. Aggiungiamo un "Positional Encoding" (per dire "questo patch era in alto a sinistra") e passiamo tutto in un Transformer Encoder standard. **Risultato:** Prestazioni superiori alle CNN se addestrato su dataset enormi, perché cattura relazioni globali fin dal primo strato.

2.9 Introduzione ai Modelli Generativi: L'Arte della Creazione

Riferimento: FDS13-intro_generation.pdf

Fino ad ora, il nostro viaggio nel Deep Learning si è concentrato sulla **Discriminazione**: data un'immagine, dire se è un gatto o un cane. Oggi entriamo nell'era della **Generazione**: vogliamo che la rete *crei* l'immagine di un gatto che non esiste.

"What I cannot create, I do not understand." – Richard Feynman Questa frase riassume lo spirito del corso: per capire davvero i dati, non basta classificarli, bisogna saperli ricostruire.

2.9.1 1. Discriminative vs Generative Modeling

Questa distinzione è domanda d'esame certa. Capiamola a fondo.

Modelli Discriminativi (Il Critico)

* **Obiettivo**: Distinguere le classi. Imparano i "confini" decisionali. * **Matematica**: Modellano la probabilità condizionata $p(y|x)$. * "Dato il pixel x , qual è la probabilità che l'etichetta y sia 'Gatto'?" * **Esempi**: CNN per classificazione, Regressione Logistica. * **Limite**: Se chiedi a una CNN "disegnami un gatto", non sa farlo. Sa solo dirti se un disegno assomiglia a un gatto.

Modelli Generativi (L'Artista)

* **Obiettivo**: Capire la struttura dei dati stessi. Imparano come sono fatti i dati. * **Matematica**: Modellano la densità di probabilità $p(x)$ (o la congiunta $p(x, y)$). * "Quanto è probabile che questa configurazione di pixel x appaia nel mondo reale?" * **Potere**: Una volta appreso $p(x)$, possiamo fare **Sampling** (campionamento): generare nuovi esempi $x_{new} \sim p(x)$ che sembrano reali ma sono inediti.

2.9.2 2. Conditional Generative Models

Spesso non vogliamo generare un dato a caso, ma un dato che rispetti una condizione.

Formula: $p(x|c)$ Dove c è il contesto (condition).

- **Text-to-Image**: c è la frase "un orsacchiotto che insegna matematica". x è l'immagine generata.
- **Image-to-Image**: c è uno schizzo in bianco e nero. x è l'immagine colorata.

2.9.3 3. La Tassonomia dei Modelli Generativi

Come facciamo a imparare $p(x)$? Le slide mostrano un albero genealogico diviso in due rami principali basati sulla **Maximum Likelihood** (Massima Verosimiglianza).

A. Explicit Density (Densità Esplicita)

Il modello definisce una formula matematica esplicita per $p(x)$ e cerca di massimizzarla.

- **Tractable Density (Trattabile):** Possiamo calcolare $p(x)$ esattamente. *Esempio:* Autoregressive Models (PixelRNN, PixelCNN).
- **Approximate Density (Approssimata):** $p(x)$ è troppo difficile da calcolare, quindi la approssimiamo. *Esempio:* Variational Autoencoders (VAE), Diffusion Models (spesso visti come approssimazione dell'evidence lower bound).

B. Implicit Density (Densità Implicita)

Il modello non definisce mai $p(x)$ esplicitamente. Non ti sa dire "questa immagine ha probabilità 0.003". Impara solo una procedura per *generare* dati (Sampling) senza conoscere la formula della densità. *Esempio:* Generative Adversarial Networks (GAN).

2.9.4 4. Autoregressive Models (PixelRNN / PixelCNN)

Questi modelli appartengono alla famiglia "Explicit Tractable Density". Usano la **Chain Rule della Probabilità** per scomporre la generazione di un'immagine in una sequenza di scelte pixel per pixel.

L'Idea Chiave: Un'immagine x non è un blocco unico, ma una sequenza di pixel $x_1, x_2, \dots, x_{n^2}$. La probabilità dell'immagine è il prodotto delle probabilità condizionate:

$$p(x) = \prod_{i=1}^{n^2} p(x_i | x_1, x_2, \dots, x_{i-1})$$

Traduzione: Il colore del pixel 50 dipende dai colori dei pixel da 1 a 49.

PixelRNN

* Genera i pixel uno alla volta, partendo dall'angolo in alto a sinistra. * Usa una **RNN (LSTM)** per ricordare il contesto dei pixel precedenti. * **Difetto:** È LENTISSIMO. Essendo sequenziale, per generare un'immagine 32×32 deve fare 1024 passaggi seriali. Non parallelizzabile.

PixelCNN

* Usa le **CNN** invece delle RNN. * Usa una **Masked Convolution**: Un filtro speciale che ha degli zeri nella parte "futura" (i pixel non ancora generati). Questo rispetta l'ordine causale: il pixel i vede solo i pixel precedenti. * **Vantaggio:** Il training è velocissimo (possiamo calcolare tutte le probabilità in parallelo perché conosciamo i pixel veri del training set). * **Svantaggio:** La generazione è ancora lenta (dobbiamo generare un pixel alla volta per poterlo dare in pasto al passo successivo).

2.9.5 5. Uno Sguardo al Futuro: VAE e Diffusion

Le slide introducono brevemente le altre famiglie (che vedremo in dettaglio nelle prossime lezioni):

* **Variational Autoencoders (VAE):** Invece di modellare i pixel direttamente, assumiamo che ci sia una variabile latente z (nascosta) che controlla il contenuto (es. z = "sorriso"). Ottimizziamo un limite inferiore della probabilità (ELBO).

* **Diffusion Models (I Re attuali):** L'idea è ispirata alla termodinamica. 1. **Forward Process (Distruzione):** Aggiungiamo rumore piano piano all'immagine finché diventa puro caos (rumore Gaussiano). 2. **Reverse Process (Creazione):** Addestriamo una rete neurale a *togliere* il rumore passo dopo passo. Per generare, partiamo dal caos e lo "scolpiamo" fino a ottenere un'immagine.

2.10 Modelli Generativi Avanzati: VAE e GAN

Riferimento: *FDS14-VAE GAN.pdf*

In questo capitolo esploriamo due approcci radicalmente diversi per generare dati:

1. **VAE (Variational Autoencoder):** Un approccio probabilistico e matematicamente rigoroso, basato sull'approssimazione della densità.
2. **GAN (Generative Adversarial Network):** Un approccio basato sulla Teoria dei Giochi, dove due reti competono tra loro.

2.10.1 1. Latent Variable Models (Il concetto di z)

Tutto parte da un'idea filosofica: le immagini che vediamo (x) sono complesse, ma sono generate da fattori nascosti (z) molto più semplici.

- x (Osservabile): Un'immagine di un volto (migliaia di pixel).
- z (Latente): Le variabili che descrivono il volto (es. età, genere, angolazione, sorriso).

L'obiettivo è imparare due cose: 1. **Generazione:** $p(x|z)$ (Dato il concetto "sorriso", genera i pixel). 2. **Inferenza:** $p(z|x)$ (Dati i pixel, capisci che c'è un "sorriso").

Il problema matematico fondamentale è che calcolare la probabilità vera dei dati $p(x)$ richiede di integrare su tutte le possibili z :

$$p(x) = \int p(x|z)p(z)dz$$

Questo integrale è **intrattabile** (impossibile da calcolare per reti neurali complesse).

2.10.2 2. Variational Autoencoders (VAE)

I VAE aggirano l'integrale intrattabile usando l'**Inferenza Variazionale**. Invece di cercare la vera distribuzione $p(z|x)$, cerchiamo di approssimarla con una distribuzione più semplice $q_\phi(z|x)$ (solitamente una Gaussiana), parametrizzata da una rete neurale.

Architettura

Un VAE è composto da due reti collegate "a clessidra":

1. **Encoder** ($q_\phi(z|x)$): Prende l'immagine x e predice **Media** (μ) e **Varianza** (σ^2) della distribuzione latente.
2. **Decoder** ($p_\theta(x|z)$): Campiona un vettore z dalla distribuzione e prova a ricostruire l'immagine originale $\hat{x}$.

Il "Reparameterization Trick" (Fondamentale)

Qui c'è un blocco tecnico. Per addestrare la rete, dobbiamo fare Backpropagation. Ma nel mezzo del VAE c'è un'operazione di **campionamento casuale** ($z \sim \mathcal{N}(\mu, \sigma)$). **Problema:** Non si può calcolare il gradiente attraverso un nodo stocastico (casuale). La casualità interrompe la catena delle derivate.

Soluzione: Spostiamo la casualità su una variabile ausiliaria esterna ϵ . Invece di campionare z direttamente, scriviamo:

$$z = \mu + \sigma \odot \epsilon, \quad \text{dove } \epsilon \sim \mathcal{N}(0, I)$$

Perché funziona? Ora z è una funzione deterministica di μ e σ (che sono i parametri da imparare). La casualità viene da ϵ , che è una costante per la rete e non richiede gradiente. *Risultato:* Possiamo derivare rispetto a μ e σ e addestrare tutto end-to-end.

La Loss Function (ELBO)

L'obiettivo è massimizzare la verosimiglianza dei dati. Matematicamente, massimizziamo il suo limite inferiore, chiamato **ELBO (Evidence Lower Bound)**. La Loss da minimizzare è:

$$\mathcal{L}_{VAE} = \underbrace{\mathbb{E}_q[\log p(x|z)]}_{\text{Reconstruction Loss}} - \underbrace{D_{KL}(q(z|x)||p(z))}_{\text{Regularization Term}}$$

Spiegazione intuitiva dei termini:

- **Reconstruction Loss (MSE):** Il Decoder deve essere bravo a ricostruire l'immagine originale. Se tolgo questo termine, la rete genera rumore casuale.
- **KL Divergence (Regolarizzazione):** Forziamo la distribuzione latente prodotta dall'Encoder ad essere vicina a una Gaussiana Standard $\mathcal{N}(0, 1)$. *Perché?* Senza questo termine, l'Encoder barerebbe nascondendo ogni immagine in un punto diverso dello spazio, creando un "dizionario" senza struttura. La KL divergence costringe lo spazio latente a essere liscio e continuo (fondamentale per generare nuove immagini interpolando punti).

2.10.3 3. Generative Adversarial Networks (GAN)

Le GAN abbandonano la stima esplicita della probabilità. Usano un approccio basato sulla **Teoria dei Giochi**.

I Giocatori

1. **Il Generatore (G):** È il "falsario". Prende un rumore casuale z e cerca di creare un'immagine x_{fake} così realistica da ingannare il discriminatore.
2. **Il Discriminatore (D):** È il "poliziotto". Prende in input un'immagine e deve classificare se è **Vera** (dal dataset) o **Falsa** (creata da G).

La Funzione Obiettivo (Minimax Game)

Questo è un gioco a somma zero. Ciò che vince l'uno, lo perde l'altro. La formula sacra delle GAN è:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]$$

Analisi Passo-Passo:

- **Punto di vista di D (Max):** Vuole massimizzare $D(x)$ (dire 1 alle immagini vere) e massimizzare $(1 - D(G(z)))$ (dire 0 alle immagini false).
- **Punto di vista di G (Min):** Vuole minimizzare il secondo termine. Vuole che $D(G(z))$ sia 1 (ingannare il poliziotto), rendendo $\log(1 - 1) = -\infty$.

Problemi di Addestramento

Addestrare le GAN è notoriamente difficile e instabile.

1. **Mode Collapse:** Il Generatore trova un'immagine che inganna bene il Discriminatore e inizia a generare *sempre e solo quella*. La diversità crolla.
2. **Vanishing Gradient:** Se il Discriminatore è troppo bravo all'inizio (distingue perfettamente il vero dal falso), il Generatore non riceve gradienti utili per migliorare.
3. **Oscillazioni:** Non stiamo cercando un minimo (come nel GD classico), ma un **Punto di Sella** (equilibrio di Nash). Le due reti possono rincorrersi all'infinito senza convergere.

2.10.4 4. Confronto Finale: VAE vs GAN

Caratteristica	VAE	GAN
Approccio	Probabilistico (Likelihood)	Teoria dei Giochi (Adversarial)
Qualità Immagini	Sfocate (Blurry)	Nitide e realistiche
Addestramento	Stabile (funziona quasi sempre)	Instabile (richiede tuning fine)
Densità $p(x)$	Esplicita (Approssimata)	Implicita (non la conosciamo)

2.11 Le Funzioni di Costo (Loss Functions)

In questa sezione analizziamo le principali funzioni di costo utilizzate nel Machine Learning, suddivise per tipologia di problema. L'obiettivo dell'algoritmo di ottimizzazione è sempre minimizzare (o massimizzare, nel caso della verosimiglianza) queste metriche.

2.11.1 1. Regressione (Predizione di valori continui)

In questo contesto, cerchiamo di predire un valore reale y_i dato un input x_i .

Mean Squared Error (MSE)

- **Utilizzo:** È la funzione di costo standard per la Regressione Lineare e le Reti Neurali per regressione. Corrisponde alla massimizzazione della verosimiglianza (MLE) assumendo che gli errori seguano una distribuzione Gaussiana.

- **Formula:**

$$\mathcal{L}(\beta) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \frac{1}{n} \sum_{i=1}^n (y_i - \beta^T x_i)^2$$

Dove $\hat{y}_i$ è la predizione del modello.

- **Derivata (Gradiente):** Per aggiornare i pesi tramite Gradient Descent, la derivata rispetto al parametro β è:

$$\nabla_{\beta} \mathcal{L} = -\frac{2}{n} \sum_{i=1}^n (y_i - \beta^T x_i) x_i$$

In pratica, è la media dell'errore (reale - predetto) moltiplicata per l'input.

Mean Absolute Error (MAE)

- **Utilizzo:** Metrica alternativa per la regressione, misura la media delle differenze assolute. È meno sensibile agli outlier rispetto all'MSE perché non eleva l'errore al quadrato.

- **Formula:**

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i|$$

2.11.2 2. Classificazione Binaria (Due classi: 0 o 1)

Qui l'output è una probabilità tra 0 e 1, ottenuta solitamente tramite una funzione di attivazione Sigmoidale (σ).

Binary Cross-Entropy (BCE) / Negative Log-Likelihood (NLL)

- **Utilizzo:** Usata per la Regressione Logistica e Reti Neurali binarie. Penalizza fortemente il modello se predice con alta confidenza la classe sbagliata (es. probabilità 0.9 per la classe 0).

- **Formula:**

$$\ell(\beta) = - \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Dove $\hat{y}_i = \sigma(\beta^T x_i)$.

- **Derivata (Gradiente):** La derivata è notevolmente semplice e simile a quella della regressione lineare (grazie alle proprietà della sigmoide):

$$\nabla_{\beta} \ell = \sum_{i=1}^n (\hat{y}_i - y_i) x_i$$

Dove $(\hat{y}_i - y_i)$ rappresenta l'errore di predizione.

2.11.3 3. Classificazione Multi-classe (K classi)

Qui l'output è un vettore di probabilità (che somma a 1), ottenuto tramite la funzione Softmax.

Cross-Entropy (Categorical) / NLL Multi-classe

- **Utilizzo:** Usata quando ci sono più di due classi (es. Iris dataset, ImageNet). Deriva dalla Negative Log-Likelihood assumendo una distribuzione multinomiale.
- **Formula:**

$$\text{NLL}(\beta) = - \sum_{i=1}^n \sum_{j=1}^K y_{ij} \log p(Y = j | x_i)$$

Dove y_{ij} è 1 se l'esempio i appartiene alla classe j , altrimenti 0 (codifica one-hot).

- **Derivata:** Il gradiente spinge i pesi β_j ad aumentare la probabilità della classe corretta e diminuire quella delle altre:

$$\nabla_{\beta_j} \text{NLL} = - \sum_{i=1}^n x_i (\mathbb{1}_{y_i=j} - p(Y = j | x_i))$$

Anche qui, la forma della derivata è intimamente riconducibile a: $\text{Input} \times (\text{Reale} - \text{Predetto})$.

2.11.4 4. Modelli Generativi (es. VAE)

Nei modelli non supervisionati come gli Autoencoder Variazionali (VAE), l'obiettivo è massimizzare l'evidenza dei dati (ELBO).

ELBO (Evidence Lower Bound)

- **Utilizzo:** Funzione obiettivo per i VAE. Si compone di due parti: una che vuole ricostruire bene i dati e una che regolarizza lo spazio latente.
- **Formula Generale:**

$$\mathcal{L}_{\text{ELBO}} = \text{Reconstruction Loss} - \text{Regularization Loss}$$

Che formalmente si esprime come:

$$\mathcal{L}_{\theta,\phi}(x) = \mathbb{E}_{q_{\phi}(z|x)}[\log p_{\theta}(x|z)] - D_{KL}(q_{\phi}(z|x)||p(z))$$

Nota: Minimizzare la Loss del VAE equivale a massimizzare l'ELBO appena descritto (per cui spesso si trova la formula moltiplicata per -1).

- **Componenti:**

1. **Reconstruction Term:** $\mathbb{E}_{q_{\phi}}[\log p_{\theta}(x|z)]$. Spesso si traduce nell'MSE (se assumiamo dati continui e Gaussiani) o nella BCE (se i dati sono binari o pixel normalizzati).
2. **KL Divergence (D_{KL}):** Misura la distanza informativa tra la distribuzione latente predetta dall'encoder $q_{\phi}(z|x)$ e una prior (solitamente Gaussiana standard). Ha una forma chiusa calcolabile analiticamente che spinge lo spazio latente ad essere continuo e centrato.

Architettura	Dominio / Input	Funzione Principale e Concetto Chiave
Linear Regression	Vettori di feature	Regressione. Predice un valore continuo minimizzando l'errore quadratico (MSE). Soluzione analitica o via Gradient Descent.
Logistic Regression	Vettori di feature	Classificazione. Predice la probabilità di appartenenza a una classe usando la funzione Sigmoidale o Softmax (per multi-classe).
Decision Trees & Random Forest	Dati Tabulari	Classificazione/Regressione Non-Parametrica. Tagliano lo spazio dei dati in regioni rettangolari. I metodi Ensemble (Forest, Boosting) riducono varianza e bias.
MLP (Multi-Layer Perceptron)	Vettori "piatti"	Approssimatore Universale. Rete densa classica. Soffre con dati ad alta dimensionalità (immagini) perché perde la struttura spaziale.
CNN (Convolutional Neural Networks)	Griglie (Immagini)	Visione Artificiale. Sfrutta la località e l'invarianza spaziale (Weight Sharing) tramite filtri convoluzionali e Pooling.
RNN (Recurrent Neural Networks)	Sequenze (Testo, Audio)	Memoria Temporale. Processa i dati sequenzialmente mantenendo uno "stato nascosto" (h_t) aggiornato ad ogni passo.
Transformers	Sequenze (NLP) e Immagini (ViT)	Relazioni Globali (Attention). Supera le RNN eliminando la ricorrenza: ogni token guarda tutti gli altri contemporaneamente (Self-Attention Q, K, V).
Autoregressive Models (PixelRNN, PixelCNN)	Immagini / Sequenze	Generazione Esplicita Trattabile. Modellano $p(x)$ scomponendola in prodotti di probabilità condizionate (regola della catena). Generano un pixel alla volta (lenti).
VAE (Variational Autoencoders)	Immagini / Dati complessi	Generazione Probabilistica (Approssimata). Mappa i dati in uno spazio latente gaussiano continuo. Ottimizza l'ELBO (Likelihood inferiore).
GAN (Generative Adversarial Networks)	Immagini / Dati complessi	Generazione Implicita. Non modellano la densità. Gioco a somma zero tra Generatore (crea falsi) e Discriminatore (smaschera falsi).

Tabella 2.1: Tassonomia Completa delle Architetture di Deep Learning (Corso FDS)

Capitolo 3

Ripasso Rapido

3.1 Concetti Fondamentali: La Chain Rule

Prima di affrontare gli esercizi, è fondamentale chiarire il motore matematico di tutto il Deep Learning: la **Chain Rule** (Regola della Catena).

3.1.1 L'Analogia del Domino

Immagina un sistema a tre stadi:

$$A \rightarrow B \rightarrow C$$

Vuoi sapere come cambia C se tocchi A . Tuttavia, A non tocca direttamente C .

1. A spinge B .
2. B spinge C .

La Chain Rule afferma che l'effetto totale è il prodotto degli effetti intermedi:

$$\frac{\partial C}{\partial A} = \underbrace{\frac{\partial C}{\partial B}}_{\text{B spinge C}} \cdot \underbrace{\frac{\partial B}{\partial A}}_{\text{A spinge B}}$$

3.1.2 L'Analogia della Cipolla (Funzioni Annidate)

Nelle reti neurali, le funzioni sono una dentro l'altra: $y = f(g(x))$. Per calcolare la derivata rispetto a x , devi "sbucciare" la funzione dall'esterno verso l'interno, moltiplicando le derivate di ogni strato.

Intuizione: La Formula Universale non Esiste

Non cercare di memorizzare una singola formula per la Backpropagation. La formula cambia se cambi i "pezzi" della rete (es. da Loss MSE a Cross-Entropy, da ReLU a Sigmoid). Il metodo, invece, è sempre lo stesso:

$$\text{Gradiente Finale} = (\text{Derivata Loss}) \cdot (\text{Derivata Attivazione}) \cdot (\text{Derivata Pesi})$$

3.2 Approfondimenti Intuitivi sulle Reti Neurali

3.2.1 1. Perché usiamo le Funzioni di Attivazione (Non-Linearità)?

Una domanda classica è: "Perché non possiamo semplicemente fare tante moltiplicazioni di matrici una dopo l'altra senza mettere ReLU o Sigmoidi in mezzo?"

Intuizione: La Trappola della Linearità

Immagina che ogni strato della rete sia un foglio di carta trasparente.

- Le operazioni lineari ($Wx + b$) possono solo **ruotare**, **scalare** o **traslare** il foglio. Non possono piegarlo.
- Se metti 100 strati lineari uno sopra l'altro, il risultato finale è equivalente a un **singolo strato lineare**.

$$W_2(W_1x) = (W_2W_1)x = W_{totale}x$$

- **L'attivazione (es. ReLU, Sigmoidi)** è ciò che ti permette di **"piegare"** il foglio.

Senza non-linearità, una rete neurale, non importa quanto profonda, si comporterebbe esattamente come una semplice Regressione Lineare. Non potrebbe risolvere problemi complessi (come riconoscere un gatto o separare dati a spirale).

3.2.2 2. Pesi (W) vs Bias (b): Cosa fanno davvero?

Matematicamente è $y = wx + b$. Ma cosa rappresentano nel decision making?

- **I Pesi (W) sono l'IMPORTANZA:** Il peso dice quanto fortemente il neurone deve reagire a un certo input. *Esempio:* Se stai decidendo se comprare una casa, il "prezzo" avrà un peso alto (w grande), mentre il "colore della porta" avrà un peso quasi nullo ($w \approx 0$). *Geometricamente:* I pesi cambiano la **pendenza** o l'orientamento della retta di separazione.
- **Il Bias (b) è la SOGLIA DI ATTIVAZIONE:** Il bias permette al neurone di attivarsi (o spegnersi) anche se l'input è zero, o di spostare la soglia di decisione. *Esempio:* Anche se una casa costa poco (input favorevole), potresti avere un "bias negativo" perché odi quel quartiere. Il bias sposta il punto in cui cambi decisione. *Geometricamente:* Il bias **sposta** la retta (su, giù, destra, sinistra) senza cambiarne l'inclinazione.

3.2.3 3. Il Learning Rate (α): L'Escursionista nella Nebbia

Il Gradient Descent è come un escursionista che deve scendere da una montagna immersa nella nebbia fitta. Non vede la valle (il minimo globale), vede solo la pendenza sotto i suoi piedi (il gradiente).

Intuizione: Scegliere il Passo Giusto

Il Learning Rate è la lunghezza del passo dell'escursionista.

- α **troppo piccolo**: L'escursionista fa passi di formica. Arriverà a valle, ma ci metterà anni (convergenza lentissima).
- α **troppo grande**: L'escursionista fa salti enormi. Potrebbe saltare oltre la valle e finire sulla montagna opposta, salendo invece di scendere (divergenza).
- α **giusto**: Passi decisi all'inizio, più cauti quando la pendenza diminuisce.

Capitolo 4

Esercizi

4.1 Esercizio Svolto di PCA (Guida Passo-Passo)

4.1.1 1. Il Dataset (Esempio)

Immaginiamo di avere 5 punti in 2 dimensioni ($N = 5, D = 2$):

$$X = \begin{bmatrix} 2 & 1 \\ 3 & 4 \\ 5 & 5 \\ 4 & 8 \\ 6 & 7 \end{bmatrix}$$

Obiettivo: Eseguire la PCA manuale, proiettare sulla prima componente e calcolare l'errore.

4.1.2 2. Centrazione dei Dati (Step Fondamentale)

La PCA lavora sulla varianza attorno alla media. Dobbiamo spostare l'origine degli assi nel baricentro dei dati. Calcoliamo le medie di colonna:

- $\mu_1 = (2 + 3 + 5 + 4 + 6)/5 = 20/5 = 4$
- $\mu_2 = (1 + 4 + 5 + 8 + 7)/5 = 25/5 = 5$

Sottraiamo il vettore media $[4, 5]$ da ogni riga di X :

$$X_{centered} = \begin{bmatrix} -2 & -4 \\ -1 & -1 \\ 1 & 0 \\ 0 & 3 \\ 2 & 2 \end{bmatrix}$$

4.1.3 3. Matrice di Covarianza

La matrice di covarianza Σ ci dice come le variabili variano insieme. La formula è $\Sigma = \frac{1}{N-1}X_c^T X_c$. Calcoliamo i prodotti scalari delle colonne:

- $\text{Var}(1): \frac{(-2)^2 + (-1)^2 + 1^2 + 0^2 + 2^2}{4} = \frac{4+1+1+0+4}{4} = \frac{10}{4} = 2.5$
- $\text{Var}(2): \frac{(-4)^2 + (-1)^2 + 0^2 + 3^2 + 2^2}{4} = \frac{16+1+0+9+4}{4} = \frac{30}{4} = 7.5$
- $\text{Cov}(1,2): \frac{(-2)(-4) + (-1)(-1) + (1)(0) + (0)(3) + (2)(2)}{4} = \frac{8+1+0+0+4}{4} = \frac{13}{4} = 3.25$

$$\Sigma = \begin{bmatrix} 2.5 & 3.25 \\ 3.25 & 7.5 \end{bmatrix}$$

Teoria: Interpretazione dell'Off-Diagonal (Domanda 8)

L'elemento fuori diagonale è $\sigma_{12} = 3.25$. **Interpretazione:** Il segno è **positivo**. Ciò significa che le due variabili sono **correlate positivamente**: quando x_1 cresce, anche x_2 tende a crescere. Se fosse stato negativo, al crescere di una l'altra sarebbe diminuita. Se fosse stato zero, sarebbero state incorrelate (indipendenti linearmente).

4.1.4 4. Autovalori e Autovettori

Dobbiamo risolvere l'equazione caratteristica $\det(\Sigma - \lambda I) = 0$. Per questo esempio (calcoli omessi per brevità, ma fattibili con la formula quadratica), otteniamo:

- $\lambda_1 \approx 9.10$ (Varianza spiegata dalla PC1)
- $\lambda_2 \approx 0.90$ (Varianza spiegata dalla PC2)

Gli autovettori corrispondenti (normalizzati) sono circa:

- $v_1 \approx [-0.44, -0.90]$ (Direzione Principale)
- $v_2 \approx [-0.90, 0.44]$ (Direzione Secondaria, ortogonale)

4.1.5 5. Varianza Spiegata (Domanda 7)

La "Explained Variance Ratio" ci dice quanta informazione conserviamo proiettando.

$$\text{Ratio PC1} = \frac{\lambda_1}{\lambda_1 + \lambda_2} = \frac{9.10}{9.10 + 0.90} = \frac{9.10}{10.0} = \mathbf{91\%}$$

Risposta: La prima componente principale cattura il 91% dell'informazione totale del dataset.

4.1.6 6. Proiezione e Ricostruzione

Proiettiamo i dati centrati sulla PC1 ($z = X_c \cdot v_1$). Esempio per il primo punto $[-2, -4]$:

$$z_1 = (-2)(-0.44) + (-4)(-0.90) = 0.88 + 3.6 = 4.48$$

Per ricostruire il punto nello spazio originale ($\hat{x}$):

$$\hat{x}_c = z_1 \cdot v_1 = 4.48 \cdot [-0.44, -0.90] \approx [-1.97, -4.03]$$

Aggiungiamo la media per tornare ai dati grezzi:

$$\hat{x} = [-1.97 + 4, -4.03 + 5] = [2.03, 0.97]$$

Il punto originale era $[2, 1]$. L'errore è piccolissimo!

4.1.7 7. MSE (Mean Squared Reconstruction Error)

L'errore quadratico medio di ricostruzione usando solo K componenti è (approssimativamente) la somma degli autovalori scartati. In modo rigoroso:

$$MSE \approx \frac{N-1}{N} \sum_{j=K+1}^D \lambda_j$$

Nel nostro caso scartiamo solo $\lambda_2 = 0.90$.

$$MSE \approx \frac{4}{5}(0.90) = \mathbf{0.72}$$

Significato: Questo è l'errore medio al quadrato che commettiamo per ogni punto sostituendo i dati originali con la loro proiezione sulla retta della PC1.

4.2 Esercizio: Backpropagation su Shallow Neural Network

Riferimento: Slide FDS08, pagine 45-55.

Analizziamo una rete con 1 layer nascosto, attivazione ReLU e Loss Binary Cross-Entropy (BCE).

4.2.1 Il Modello (Forward Pass)

1. **Input:** X (matrice $N \times D_{in}$)

2. **Hidden Layer:**

$$h = \text{ReLU}(XW_1)$$

Dove W_1 sono i pesi del primo livello.

3. **Output Layer:**

$$z = hW_2 + b$$

$$\hat{y} = \sigma(z) \quad (\text{Sigmoide})$$

4. **Loss (BCE):** Misura l'errore tra $\hat{y}$ e il target reale y .

4.2.2 Backward Pass: Calcolo dei Gradienti

Passo 1: Errore sull'Output ($\frac{\partial \mathcal{L}}{\partial z}$)

Vogliamo sapere come cambia l'errore rispetto al pre-attivazione z . La combinazione della derivata della BCE e della Sigmoide porta a una semplificazione notevole:

$$\frac{\partial \mathcal{L}}{\partial z} = (\hat{y} - y)$$

Questo è il segnale di errore "puro" che viaggia all'indietro.

Passo 2: Gradiente su W_2 e b

Ora portiamo l'errore $(\hat{y} - y)$ ai pesi W_2 .

$$\frac{\partial \mathcal{L}}{\partial W_2} = h^T \cdot (\hat{y} - y)$$

Per il bias b , sommiamo gli errori su tutti gli esempi: $\sum(\hat{y} - y)$.

Passo 3: Gradiente su W_1 (Il passaggio difficile)

Dobbiamo portare l'errore attraverso la ReLU e attraverso W_1 . La formula finale è:

$$\frac{\partial \mathcal{L}}{\partial W_1} = X^T \cdot \left[\underbrace{(\hat{y} - y) \cdot W_2^T}_{\text{Errore retro-propagato}} \odot \underbrace{1_{(XW_1 > 0)}}_{\text{ReLU Mask}} \right]$$

Intuizione: Spiegazione Dimensionale

Perché usiamo le trasposte (T)? Per far combaciare le dimensioni delle matrici.

- W_2^T : L'errore fluisce indietro (Output $\rightarrow$ Hidden), quindi dobbiamo "invertire" la matrice dei pesi W_2 .
- **ReLU Mask**: La derivata della ReLU è 1 se il valore è positivo, 0 se negativo. Questa maschera "spegne" il gradiente per i neuroni inattivi.
- X^T : L'aggiornamento dei pesi dipende dall'input. Se l'input era forte, il peso deve cambiare molto.

4.3 Esercizi di Ottimizzazione: Spiegazione Dettagliata

Riferimento: *ex_optimization.pdf*

In questa sezione esploriamo come i computer calcolano le derivate (fondamentali per il training) e come usano queste derivate per minimizzare l'errore.

4.3.1 Esercizio 1: Forward Mode AD (Differenziazione Automatica in Avanti)

Obiettivo: Calcolare la derivata di una funzione complessa scomponendola in pezzi elementari, procedendo dall'inizio alla fine (Forward). **Funzione:** $h(w) = \ln(w) + e^w \cos(w)$.

Procedimento: In Forward Mode, ogni volta che calcoliamo il valore di un nodo, calcoliamo immediatamente anche la sua derivata rispetto all'input w . Non dobbiamo aspettare la fine.

1. **Nodo a (Logaritmo):** $a = \ln(w)$.
 - *Derivata:* Sappiamo che la derivata di $\ln(x)$ è $1/x$.
 - $\dot{a} = \frac{da}{dw} = \frac{1}{w}$.
2. **Nodo b (Esponenziale):** $b = e^w$.
 - *Derivata:* L'esponenziale è l'unica funzione che è derivata di se stessa.
 - $\dot{b} = \frac{db}{dw} = e^w$.
3. **Nodo c (Coseno):** $c = \cos(w)$.
 - *Derivata:* La derivata del coseno è meno seno.
 - $\dot{c} = \frac{dc}{dw} = -\sin(w)$.
4. **Nodo d (Prodotto):** $d = b \cdot c = e^w \cos(w)$.
 - *Regola:* Qui usiamo la **Regola del Prodotto**: $(f \cdot g)' = f'g + fg'$.
 - $\dot{d} = \dot{b} \cdot c + b \cdot \dot{c} = e^w \cos(w) + e^w(-\sin(w)) = e^w(\cos(w) - \sin(w))$.
5. **Nodo Finale h (Somma):** $h = a + d$.
 - *Regola:* La derivata di una somma è la somma delle derivate.
 - $\dot{h} = \dot{a} + \dot{d}$.

Risultato Finale:

$$\frac{dh}{dw} = \frac{1}{w} + e^w(\cos(w) - \sin(w))$$

4.3.2 Esercizio 2: Reverse Mode AD (Backpropagation Manuale)

Obiettivo: Questo è il metodo usato dalle Reti Neurali. Prima calcoliamo il risultato finale (Forward), poi torniamo indietro (Reverse) per capire di chi è la "colpa" del risultato. **Funzione:** $f(x, y) = (x + y)(xy)$ nei punti $x = 2, y = 3$.

Fase 1: Forward Pass (Calcoliamo i valori) Costruiamo il grafo passo dopo passo:

- Calcoliamo la somma $s = x + y = 2 + 3 = 5$.
- Calcoliamo il prodotto $p = x \cdot y = 2 \cdot 3 = 6$.
- Calcoliamo l'output finale $f = s \cdot p = 5 \cdot 6 = 30$.

Fase 2: Reverse Pass (Calcoliamo i gradienti) Partiamo dalla fine (f) e torniamo verso gli input (x, y). Ci chiediamo: "Se cambio leggermente questo nodo, quanto cambia f ?"

1. **Gradienti sui nodi intermedi (s e p):** La funzione finale è un prodotto $f = s \cdot p$.
 - Rispetto a s : Se $f = s \cdot 6$, la derivata è 6. $\rightarrow \frac{\partial f}{\partial s} = p = 6$.
 - Rispetto a p : Se $f = 5 \cdot p$, la derivata è 5. $\rightarrow \frac{\partial f}{\partial p} = s = 5$.
2. **Gradienti sugli input (x e y):** Qui bisogna fare attenzione. La variabile x influenza f attraverso **DUE strade**: influenza la somma s E influenza il prodotto p . Dobbiamo sommare entrambi i contributi (Chain Rule Multivariata).

$$\frac{\partial f}{\partial x} = \underbrace{\frac{\partial f}{\partial s} \cdot \frac{\partial s}{\partial x}}_{\text{Via somma}} + \underbrace{\frac{\partial f}{\partial p} \cdot \frac{\partial p}{\partial x}}_{\text{Via prodotto}}$$

- $\frac{\partial s}{\partial x} = 1$ (derivata di $x + y$ rispetto a x).
- $\frac{\partial p}{\partial x} = y = 3$ (derivata di xy rispetto a x).

Calcolo per X:

$$\frac{\partial f}{\partial x} = (6 \cdot 1) + (5 \cdot 3) = 6 + 15 = 21$$

Calcolo per Y: Similmente per y :

$$\frac{\partial f}{\partial y} = (6 \cdot 1) + (5 \cdot 2) = 6 + 10 = 16$$

4.3.3 Esercizio 3: Backpropagation con Tanh (Funzione di Attivazione)

Obiettivo: Applicare la Backpropagation a una funzione non-lineare tipica delle reti neurali ($\tanh$). **Funzione:** $z = \tanh(xy + 1)$ con input $x = 1, y = 2$.

1. **Grafo e Forward Pass:** Immaginiamo la funzione come strati di una cipolla:

- Nucleo: $u = x \cdot y = 1 \cdot 2 = 2$.
- Strato intermedio: $v = u + 1 = 3$.
- Strato esterno: $z = \tanh(v) = \tanh(3) \approx 0.995$.

2. **Reverse Pass (Sbucciamo la cipolla):** Per trovare come x influenza z , dobbiamo moltiplicare le derivate partendo dall'esterno.

- **Step 1 (Esterno):** Derivata di $\tanh(v)$. La regola è: $\frac{d}{dv} \tanh(v) = 1 - \tanh^2(v) = 1 - z^2$.

$$\frac{\partial z}{\partial v} = 1 - (0.995)^2 \approx 0.01$$

Significato: Poiché $\tanh(3)$ è quasi piatto (siamo nella zona di saturazione), il gradiente è piccolissimo. Cambiare l'input avrà poco effetto.

- **Step 2 (Intermedio):** Derivata di $v = u + 1$. La derivata è 1 (il $+1$ è una costante).

$$\frac{\partial z}{\partial u} = \frac{\partial z}{\partial v} \cdot 1 = 0.01$$

- **Step 3 (Nucleo):** Derivata di $u = x \cdot y$. Per arrivare a x , deriviamo xy rispetto a x , che fa y .

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial u} \cdot y = 0.01 \cdot 2 = 0.02$$

Similmente per y :

$$\frac{\partial z}{\partial y} = \frac{\partial z}{\partial u} \cdot x = 0.01 \cdot 1 = 0.01$$

4.3.4 Esercizio 5: Gradient Descent (1 Variabile)

Obiettivo: Simulare manualmente l'apprendimento. Vogliamo trovare il minimo della funzione spostandoci passo dopo passo. **Funzione Costo:** $g(w) = (w - 3)^2$. Minimo ovvio a $w = 3$. **Parametri:** Partiamo da $w_0 = 0$. Learning Rate $\alpha = 0.1$.

Logica: Il gradiente ci dice la pendenza.

- Se la pendenza è negativa (discesa a destra), dobbiamo aumentare w .
- Se la pendenza è positiva (salita a destra), dobbiamo diminuire w .
- La formula $w_{new} = w - \alpha \cdot \text{Gradiente}$ fa questo automaticamente.

Calcoli Passo-Passo:

- **Step 0:** Siamo a $w = 0$.
 - Gradiente: $g'(0) = 2(0 - 3) = -6$ (Pendenza ripida verso il basso).
 - Passo: $-\alpha \cdot (-6) = -0.1 \cdot -6 = +0.6$.
 - Nuovo $w_1 = 0 + 0.6 = 0.6$.
- **Step 1:** Siamo a $w = 0.6$.
 - Gradiente: $g'(0.6) = 2(0.6 - 3) = 2(-2.4) = -4.8$ (Pendenza ancora negativa, ma meno ripida).
 - Passo: $-0.1 \cdot (-4.8) = +0.48$.
 - Nuovo $w_2 = 0.6 + 0.48 = 1.08$.

Risultato: Ci stiamo avvicinando a 3 ($0 \rightarrow 0.6 \rightarrow 1.08$).

4.3.5 Esercizio 6: Gradient Descent (2 Variabili)

Obiettivo: Capire cosa succede quando le dimensioni hanno scale diverse (problema delle "valli strette"). **Funzione:** $g(w_1, w_2) = w_1^2 + 2w_2^2$. Notare il **2** davanti a w_2 : significa che la funzione è più ripida lungo l'asse w_2 (è un'ellisse stretta, non un cerchio perfetto).

Calcoli:

- Gradiente: $\nabla g = [2w_1, 4w_2]^T$. (Notare che il gradiente su w_2 è doppio rispetto a w_1).
- Partiamo da $(2, 1)$ con $\alpha = 0.2$.
- **Aggiornamento w_1 :** $w_1 \leftarrow w_1 - 0.2(2w_1) = w_1 - 0.4w_1 = 0.6w_1$. Ogni passo riduce la distanza dal centro del 40%.
- **Aggiornamento w_2 :** $w_2 \leftarrow w_2 - 0.2(4w_2) = w_2 - 0.8w_2 = 0.2w_2$. Ogni passo riduce la distanza dal centro dell'80%!

Conclusione Importante: Dopo un solo passo:

- w_1 passa da 2 a 1.2.
- w_2 passa da 1 a 0.2.

La coordinata w_2 crolla verso lo zero molto più velocemente. Questo sbilanciamento può causare oscillazioni ("zig-zag") in problemi più complessi. Ecco perché nel Deep Learning usiamo tecniche come la **Batch Normalization** o ottimizzatori come **Adam**, per rendere la discesa più uniforme in tutte le direzioni.

4.4 Esercizi Svolti: Regressione e Classificazione

Riferimento Completo: Regression exercises.pdf

In questa sezione analizziamo nel dettaglio ogni singolo calcolo richiesto dagli esercizi, giustificando ogni passaggio teorico.

4.4.1 Esercizio 1: Regressione Lineare (Equazioni Normali)

Dati: Abbiamo tre punti: $(1, 2), (2, 3), (3, 5)$. **Modello:** $y = a + bx$ (o in notazione vettoriale $y = \beta_0 + \beta_1 x$).

(a) Calcolo dei Parametri Ottimali

Vogliamo risolvere il sistema $X^T X \beta = X^T Y$.

1. Costruzione delle Matrici La matrice X (Design Matrix) deve avere una colonna di 1 per l'intercetta (bias) e una colonna con i valori x .

$$X = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix}, \quad Y = \begin{bmatrix} 2 \\ 3 \\ 5 \end{bmatrix}$$

2. Matrice di Gram ($X^T X$)

$$X^T X = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix} = \begin{bmatrix} 3 & 6 \\ 6 & 14 \end{bmatrix}$$

Dettaglio calcolo:

- Elemento (1,1): $1 \cdot 1 + 1 \cdot 1 + 1 \cdot 1 = 3$ (N)
- Elemento (1,2): $1 \cdot 1 + 1 \cdot 2 + 1 \cdot 3 = 6$ ($\sum x$)
- Elemento (2,2): $1^2 + 2^2 + 3^2 = 1 + 4 + 9 = 14$ ($\sum x^2$)

3. Vettore Correlazione ($X^T Y$)

$$X^T Y = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \end{bmatrix} \begin{bmatrix} 2 \\ 3 \\ 5 \end{bmatrix} = \begin{bmatrix} 2 + 3 + 5 \\ 2 + 6 + 15 \end{bmatrix} = \begin{bmatrix} 10 \\ 23 \end{bmatrix}$$

4. Inversione e Soluzione Il determinante di $X^T X$ è $D = (3)(14) - (6)(6) = 42 - 36 = 6$.
L'inversa è:

$$(X^T X)^{-1} = \frac{1}{6} \begin{bmatrix} 14 & -6 \\ -6 & 3 \end{bmatrix}$$

Moltiplichiamo per $X^T Y$:

$$\beta = \frac{1}{6} \begin{bmatrix} 14 & -6 \\ -6 & 3 \end{bmatrix} \begin{bmatrix} 10 \\ 23 \end{bmatrix} = \frac{1}{6} \begin{bmatrix} 140 - 138 \\ -60 + 69 \end{bmatrix} = \frac{1}{6} \begin{bmatrix} 2 \\ 9 \end{bmatrix}$$

$$\hat{a} = 1/3 \approx 0.333, \quad \hat{b} = 3/2 = 1.5$$

Modello Finale: $\hat{y} = 0.333 + 1.5x$.

(b) Confronto Log-Likelihood (Modello Fittato vs Modello Zero)

Domanda: Assumendo rumore Gaussiano con varianza $\sigma^2 = 1$, calcolare la Log-Likelihood del modello appena trovato e confrontarla con il modello nullo ($a = b = 0$).

Teoria: La Log-Likelihood per la regressione lineare è:

$$\ln L = -\frac{N}{2} \ln(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Poiché ci interessa solo il confronto, possiamo ignorare la costante $-\frac{N}{2} \ln(2\pi)$ (che è uguale per entrambi) e concentrarci sul termine di errore (SSE).

$$\text{Score} \approx -\frac{1}{2} \sum (y_i - \hat{y}_i)^2$$

1. Modello Fittato (M_1) Calcoliamo gli errori quadratici per ogni punto:

- $x = 1, y = 2$: $\hat{y} = 0.33 + 1.5(1) = 1.83$. Errore Sq: $(2 - 1.83)^2 = 0.0289$.
- $x = 2, y = 3$: $\hat{y} = 0.33 + 1.5(2) = 3.33$. Errore Sq: $(3 - 3.33)^2 = 0.1089$.
- $x = 3, y = 5$: $\hat{y} = 0.33 + 1.5(3) = 4.83$. Errore Sq: $(5 - 4.83)^2 = 0.0289$.

Somma Errori Quadratici (SSE_1) = $0.0289 + 0.1089 + 0.0289 \approx 0.167$. **Log-Likelihood (Approx):** $-0.167/2 = -0.0835$.

2. Modello Zero (M_0) Il modello predice sempre 0 ($\hat{y} = 0$).

- $y = 2$: Errore Sq $2^2 = 4$.
- $y = 3$: Errore Sq $3^2 = 9$.
- $y = 5$: Errore Sq $5^2 = 25$.

Somma Errori Quadratici (SSE_0) = $4 + 9 + 25 = 38$. **Log-Likelihood (Approx):** $-38/2 = -19.0$.

Conclusione: Il modello fittato ha una likelihood molto maggiore (più vicina a zero) rispetto al modello nullo.

4.4.2 Esercizio 2: Regressione Lineare (Gradient Descent)

Dati: Due punti $\{(1, 3), (2, 5)\}$. **Loss Function:** $MSE = \frac{1}{N} \sum (y - \hat{y})^2$. **Inizializzazione:** $\beta_0 = 1, \beta_1 = 1$. Learning Rate $\alpha = 0.1$.

1. Forward Pass (Predizione) Calcoliamo le stime con i pesi attuali:

- Punto 1 ($x = 1$): $\hat{y}_1 = 1 + 1(1) = 2$. (Target $y = 3$). Residuo $e_1 = 3 - 2 = 1$.
- Punto 2 ($x = 2$): $\hat{y}_2 = 1 + 1(2) = 3$. (Target $y = 5$). Residuo $e_2 = 5 - 3 = 2$.

2. Calcolo dei Gradienti Le formule fornite sono:

$$\frac{\partial E}{\partial \beta_0} = -\frac{2}{N} \sum (y_i - \hat{y}_i)$$

$$\frac{\partial E}{\partial \beta_1} = -\frac{2}{N} \sum x_i(y_i - \hat{y}_i)$$

Sostituiamo i valori ($N = 2$):

- Gradiente β_0 : $-\frac{2}{2}(e_1 + e_2) = -(1 + 2) = -3$.
- Gradiente β_1 : $-\frac{2}{2}(x_1e_1 + x_2e_2) = -(1 \cdot 1 + 2 \cdot 2) = -(1 + 4) = -5$.

3. Aggiornamento Pesi (One Step)

$$\beta_0^{new} = \beta_0 - \alpha(\text{Grad}_0) = 1 - 0.1(-3) = 1 + 0.3 = 1.3$$

$$\beta_1^{new} = \beta_1 - \alpha(\text{Grad}_1) = 1 - 0.1(-5) = 1 + 0.5 = 1.5$$

4.4.3 Esercizio 3: Binary Logistic Regression (Gradient Descent)

Testo: Abbiamo un dataset unidimensionale con intercetta implicita nel modello β_0 .

- **Dati:** $x \in \{0, 1, 2, 3\}$ con etichette corrispondenti $y \in \{0, 0, 1, 1\}$.
- **Modello:** $p(y = 1|x) = \sigma(\beta_0 + \beta_1 x)$, dove $\sigma(z) = \frac{1}{1+e^{-z}}$.
- **Loss:** Negative Log-Likelihood (NLL).
- **Inizializzazione:** Parametri $(\beta_0, \beta_1) = (0, 0)$. Learning Rate $\alpha = 0.5$.
- **Obiettivo:** Eseguire un passo di aggiornamento (One Update Step).

Svolgimento Passo-Passo

Passo 1: Predizioni Iniziali (Forward Pass) Con i parametri iniziali $\beta_0 = 0, \beta_1 = 0$, il logit z è:

$$z_i = \beta_0 + \beta_1 x_i = 0 + 0 \cdot x_i = 0 \quad \forall i$$

La probabilità predetta è:

$$\hat{y}_i = \sigma(0) = \frac{1}{1 + e^0} = \frac{1}{2} = 0.5$$

Il modello è inizialmente incerto (50%) su tutti i campioni.

Passo 2: Calcolo degli Errori L'errore è la differenza tra la probabilità predetta e l'etichetta vera ($\hat{y}_i - y_i$).

- Campione 1 ($x = 0, y = 0$): $0.5 - 0 = +0.5$
- Campione 2 ($x = 1, y = 0$): $0.5 - 0 = +0.5$
- Campione 3 ($x = 2, y = 1$): $0.5 - 1 = -0.5$
- Campione 4 ($x = 3, y = 1$): $0.5 - 1 = -0.5$

Passo 3: Calcolo dei Gradienti Le derivate della NLL rispetto ai parametri sono la somma degli errori (per l'intercetta) e la somma degli errori pesati per l'input (per la pendenza).

$$\frac{\partial \mathcal{L}}{\partial \beta_0} = \sum_{i=1}^4 (\hat{y}_i - y_i) = 0.5 + 0.5 - 0.5 - 0.5 = 0$$

$$\frac{\partial \mathcal{L}}{\partial \beta_1} = \sum_{i=1}^4 x_i (\hat{y}_i - y_i)$$

Svolgiamo la somma per β_1 :

$$\begin{aligned} & (0 \cdot 0.5) + (1 \cdot 0.5) + (2 \cdot -0.5) + (3 \cdot -0.5) \\ &= 0 + 0.5 - 1.0 - 1.5 = -2.0 \end{aligned}$$

Passo 4: Aggiornamento dei Parametri Regola di update: $\beta^{new} = \beta^{old} - \alpha \cdot \nabla \mathcal{L}$.

- $\beta_0^{new} = 0 - 0.5(0) = 0$
- $\beta_1^{new} = 0 - 0.5(-2.0) = 0 + 1.0 = 1.0$

Risultato: I nuovi parametri sono $(\beta_0, \beta_1) = (0, 1.0)$. *Interpretazione:* Il modello ha imparato che all'aumentare di x , la probabilità di classe 1 aumenta (pendenza positiva).

4.4.4 Esercizio 4: Multi-Class Logistic Regression (Softmax)

Testo: Stiamo addestrando un classificatore a 3 classi ($K = 3$).

- **Dato:** Un singolo esempio (x, y) con $x = [1, 5]^T$ (bias incluso implicitamente nel vettore o non presente) e $y = 1$ (Classe 1).
- **Parametri Correnti:** $\beta_1 = [0, 0.5]^T$, $\beta_2 = [1, 0]^T$, $\beta_3 = [-1, 0.2]^T$.
- **Learning Rate:** $\alpha = 0.5$.
- **Obiettivo:** Un passo di Gradient Descent.

Svolgimento Passo-Passo

Passo 1: Calcolo dei Logits (Scores) Calcoliamo $z_k = \beta_k^T x$ per ogni classe $k \in \{1, 2, 3\}$.

- Classe 1: $[0, 0.5] \cdot [1, 5]^T = (0 \cdot 1) + (0.5 \cdot 5) = 2.5$
- Classe 2: $[1, 0] \cdot [1, 5]^T = (1 \cdot 1) + (0 \cdot 5) = 1.0$
- Classe 3: $[-1, 0.2] \cdot [1, 5]^T = (-1 \cdot 1) + (0.2 \cdot 5) = -1 + 1 = 0.0$

Passo 2: Calcolo Probabilità (Softmax)

$$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

Denominatore: $e^{2.5} + e^{1.0} + e^0 \approx 12.182 + 2.718 + 1 = 15.9$.

- $p_1 = 12.182/15.9 \approx 0.766$
- $p_2 = 2.718/15.9 \approx 0.171$
- $p_3 = 1.0/15.9 \approx 0.063$

Passo 3: Calcolo dei Gradienti Il gradiente per la classe k è: $\nabla_{\beta_k} = (p_k - \mathbb{I}(y = k)) \cdot x$. L'etichetta vera è $y = 1$.

- **Classe 1 (Vera):** $(p_1 - 1)x = (0.766 - 1)x = -0.234 \cdot [1, 5]^T = [-0.234, -1.17]$
- **Classe 2 (Falsa):** $(p_2 - 0)x = 0.171 \cdot [1, 5]^T = [0.171, 0.855]$
- **Classe 3 (Falsa):** $(p_3 - 0)x = 0.063 \cdot [1, 5]^T = [0.063, 0.315]$

Passo 4: Aggiornamento Parametri $\beta^{new} = \beta^{old} - 0.5 \cdot \text{Gradiente}$.

- $\beta_1^{new} = [0, 0.5] - 0.5[-0.234, -1.17] = [0 + 0.117, 0.5 + 0.585] = [0.117, 1.085]$
- $\beta_2^{new} = [1, 0] - 0.5[0.171, 0.855] = [1 - 0.0855, 0 - 0.4275] = [0.915, -0.428]$
- $\beta_3^{new} = [-1, 0.2] - 0.5[0.063, 0.315] = [-1 - 0.0315, 0.2 - 0.1575] = [-1.032, 0.043]$

4.4.5 Esercizio 5: ROC Curve e AUC

Testo: 8 campioni ordinati per probabilità decrescente. Totale Positivi (P) = 5. Totale

Sample	A	B	C	D	E	F	G	H
Prob	0.95	0.90	0.85	0.80	0.70	0.60	0.40	0.20
Label (Y)	1	0	1	1	0	1	0	1

Negativi (N) = 3. Passo Verticale (TPR) = $1/5 = 0.2$. Passo Orizzontale (FPR) = $1/3 \approx 0.33$.

(a) Coordinate della Curva ROC

Partiamo da (0, 0). Per ogni campione, se è 1 saliamo, se è 0 andiamo a destra.

1. **A (1):** Su $\rightarrow (0, 0.2)$
2. **B (0):** Destra $\rightarrow (0.33, 0.2)$
3. **C (1):** Su $\rightarrow (0.33, 0.4)$
4. **D (1):** Su $\rightarrow (0.33, 0.6)$
5. **E (0):** Destra $\rightarrow (0.66, 0.6)$
6. **F (1):** Su $\rightarrow (0.66, 0.8)$
7. **G (0):** Destra $\rightarrow (1.0, 0.8)$
8. **H (1):** Su $\rightarrow (1.0, 1.0)$

(b) Calcolo AUC (Regola del Trapezio/Rettangoli)

Calcoliamo l'area sotto i segmenti orizzontali (quando ci muoviamo a destra).

- **Sotto B:** Base $1/3$, Altezza 0.2 . Area = $(1/3) \cdot (1/5) = 1/15 \approx 0.067$.
- **Sotto E:** Base $1/3$, Altezza 0.6 (dato da A+C+D). Area = $(1/3) \cdot (3/5) = 3/15 = 0.200$.
- **Sotto G:** Base $1/3$, Altezza 0.8 (dato da A+C+D+F). Area = $(1/3) \cdot (4/5) = 4/15 \approx 0.267$.

Somma Totale AUC:

$$AUC = \frac{1}{15} + \frac{3}{15} + \frac{4}{15} = \frac{8}{15} \approx \mathbf{0.533}$$

(c) Verifica con Confronto a Coppie

Contiamo quante volte un Positivo ha uno score maggiore di un Negativo. Coppie Possibili: $5 \times 3 = 15$.

- **A (0.95):** Vince su B, E, G (3 vittorie).
- **C (0.85):** Vince su E, G (perde con B). (2 vittorie).
- **D (0.80):** Vince su E, G (perde con B). (2 vittorie).
- **F (0.60):** Vince su G (perde con B, E). (1 vittoria).
- **H (0.20):** Non vince nessuno. (0 vittorie).

Totale Vittorie: $3 + 2 + 2 + 1 + 0 = 8$.

$$AUC = \frac{\text{Vittorie}}{\text{Totale Coppie}} = \frac{8}{15} \approx 0.533$$

Il risultato è confermato.

4.5 Algoritmi Classici: Alberi, KNN, Random Forest

Riferimento: *exercise_knn_tree_rf.pdf*

4.5.1 Albero Decisionale

L'obiettivo è dividere i dati in gruppi "puri". **Esempio:** Dividere Frutta (Mele/Pere) basandosi su Peso e Colore.

- **Split 1 (Peso):** Se peso $> 160g$, sono tutte Mele (Nodo puro). Se $\leq 160g$, è misto.
- **Split 2 (Colore):** Nel gruppo misto, se Colore è Verde sono Pere, se Rosso sono Mele.

Un nuovo frutto viene classificato seguendo questo percorso dall'alto verso il basso.

4.5.2 k-Nearest Neighbors (k-NN)

Non costruisce un modello, ma memorizza i dati. **Procedura per classificare un nuovo punto X:**

1. Calcola la **Distanza Euclidea** tra X e tutti i punti noti.

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

2. Seleziona i k **vicini più prossimi** (quelli con distanza minore).
3. Fai votare la maggioranza.

Attenzione

Se le scale delle feature sono diverse (es. Peso 150g vs Colore 0/1), la caratteristica con numeri più grandi dominerà il calcolo della distanza. È essenziale normalizzare i dati!

4.5.3 Random Forest

Concetto: Invece di un solo albero (che può sbagliare se i dati sono rumorosi), ne costruiamo tanti diversi.

1. Ogni albero viene addestrato su un sottoinsieme casuale dei dati (Bootstrap).
2. Ogni albero vede solo un sottoinsieme casuale delle feature.
3. Per predire, ogni albero dà il suo voto. Vince la maggioranza.

Questo metodo ("Ensemble") riduce la varianza ed è molto più robusto di un singolo albero decisionale.

4.6 Parte 1: Convoluzione 1D

Teoria: Cos'è la Convoluzione Discreta?

Matematicamente, la convoluzione tra un segnale di input x e un filtro (kernel) k è definita come:

$$(x * k)[n] = \sum_m x[m] \cdot k[n - m]$$

Questo implica che il kernel venga **"ribaltato"** (flipped) di 180 gradi prima di scorrere sull'input.

Nota: In molte librerie di Deep Learning (es. PyTorch, TensorFlow), l'operazione implementata è tecnicamente una *Cross-Correlation* (il kernel non viene ribaltato). Tuttavia, negli esercizi teorici "classici" (come questo corso), si richiede spesso l'operazione matematica formale.

Parametri:

- **Input:** Sequenza di dati.
- **Kernel (Filtro):** Pesi da applicare.
- **Stride (S):** Il passo di scorrimento (di quanto ci spostiamo a ogni calcolo).
- **Padding (P):** Zeri aggiunti ai bordi. "Valid" significa $P = 0$ (nessun padding).

Testo dell'Esercizio

Input: $x = [2, 5, 3, 4, 1, 6]$

Kernel: $k = [1, 0, -1]$

Configurazione: Padding = Valid (0), Stride = 1.

Svolgimento

Passo 1: Ribaltamento del Kernel

Poiché eseguiamo una convoluzione matematica, ribaltiamo il kernel k :

$$k_{flip} = [-1, 0, 1]$$

Passo 2: Scorrimento (Sliding Window)

Facciamo scorrere k_{flip} sull'input x calcolando il prodotto scalare (dot product) ad ogni passo.

Posizione 1 (Indici 0-2): Input window: $[2, 5, 3]$

$$y_0 = (2 \cdot -1) + (5 \cdot 0) + (3 \cdot 1) = -2 + 0 + 3 = \mathbf{1}$$

Posizione 2 (Indici 1-3): Input window: $[5, 3, 4]$

$$y_1 = (5 \cdot -1) + (3 \cdot 0) + (4 \cdot 1) = -5 + 0 + 4 = \mathbf{-1}$$

Posizione 3 (Indici 2-4): Input window: $[3, 4, 1]$

$$y_2 = (3 \cdot -1) + (4 \cdot 0) + (1 \cdot 1) = -3 + 0 + 1 = \mathbf{-2}$$

Posizione 4 (Indici 3-5): Input window: $[4, 1, 6]$

$$y_3 = (4 \cdot -1) + (1 \cdot 0) + (6 \cdot 1) = -4 + 0 + 6 = \mathbf{2}$$

Risultato Finale

L'output della convoluzione è il vettore:

$$Y = [1, -1, -2, 2]$$

4.7 Parte 2: Convoluzione 2D

Teoria: Convoluzione 2D e Stride

Il concetto è identico all'1D, ma ci muoviamo su due assi. Se **Stride** = **2**, spostiamo la finestra di 2 posizioni alla volta (saltiamo una colonna/riga). Anche qui, per rigore matematico, **ruotiamo il kernel di 180 gradi** (che equivale a ribaltarlo sia orizzontalmente che verticalmente).

Testo dell'Esercizio

Input Matrix (4×4):

$$X = \begin{bmatrix} 3 & 1 & 2 & 0 \\ 1 & 0 & 4 & 2 \\ 2 & 1 & 0 & 3 \\ 4 & 2 & 1 & 0 \end{bmatrix}$$

Kernel (2×2):

$$K = \begin{bmatrix} 1 & -1 \\ 0 & 2 \end{bmatrix}$$

Configurazione: Stride = 2, Padding = Valid.

Svolgimento

Passo 1: Ribaltamento del Kernel

Ruotiamo K di 180 gradi:

$$K_{flip} = \begin{bmatrix} 2 & 0 \\ -1 & 1 \end{bmatrix}$$

Verifica: In alto a sx c'era 1, ora va in basso a dx. In basso a dx c'era 2, ora va in alto a sx.

Passo 2: Scorrimento con Stride 2

Dobbiamo applicare il kernel 2×2 sulla matrice X . Poiché Stride=2, partiremo da $(0, 0)$, poi ci sposteremo a $(0, 2)$ (saltando la colonna 1), poi a $(2, 0)$ (saltando la riga 1) e infine a $(2, 2)$.

1. **Posizione Top-Left (Righe 0-1, Col 0-1):** Sottomatrice: $\begin{bmatrix} 3 & 1 \\ 1 & 0 \end{bmatrix}$ Calcolo:

$$(3 \cdot 2) + (1 \cdot 0) + (1 \cdot -1) + (0 \cdot 1) = 6 + 0 - 1 + 0 = 5$$

2. **Posizione Top-Right (Righe 0-1, Col 2-3):** Sottomatrice: $\begin{bmatrix} 2 & 0 \\ 4 & 2 \end{bmatrix}$ Calcolo:

$$(2 \cdot 2) + (0 \cdot 0) + (4 \cdot -1) + (2 \cdot 1) = 4 + 0 - 4 + 2 = 2$$

3. **Posizione Bottom-Left (Righe 2-3, Col 0-1):** Sottomatrice: $\begin{bmatrix} 2 & 1 \\ 4 & 2 \end{bmatrix}$ Calcolo:

$$(2 \cdot 2) + (1 \cdot 0) + (4 \cdot -1) + (2 \cdot 1) = 4 + 0 - 4 + 2 = 2$$

4. **Posizione Bottom-Right (Righe 2-3, Col 2-3):** Sottomatrice: $\begin{bmatrix} 0 & 3 \\ 1 & 0 \end{bmatrix}$ Calcolo:

$$(0 \cdot 2) + (3 \cdot 0) + (1 \cdot -1) + (0 \cdot 1) = 0 + 0 - 1 + 0 = -1$$

Risultato Finale

$$Y = \begin{bmatrix} 5 & 2 \\ 2 & -1 \end{bmatrix}$$

4.8 Parte 3: Backpropagation (Rete Neurale)

Teoria: Il Grafo Computazionale

Per calcolare i gradienti in una rete neurale, costruiamo un grafo dove i nodi sono le operazioni e gli archi portano i dati. La **Backpropagation** è l'applicazione della Chain Rule (Regola della Catena) partendo dalla Loss finale tornando indietro ai pesi.

Riferimento: exercise_convolution.pdf (Pag. 4) e relative soluzioni

In questo esercizio eseguiamo un'iterazione completa di addestramento (Forward + Backward) su una piccola rete neurale.

4.8.1 1. Setup della Rete

Architettura:

- **Input Layer:** Nodi 1 e 2. Valori: $x_1 = 1.0, x_2 = 0.1$.
- **Hidden Layer:** Nodi 3 e 4. Funzione di attivazione: $\tanh(\cdot)$.
- **Output Layer:** Nodo 5. Funzione di attivazione: Lineare (identità).
- **Target (y):** 0.5.
- **Loss Function:** Euclidean Loss $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$.

- **Learning Rate (η):** -0.2 . *Nota:* Il segno negativo indica che la regola di aggiornamento usata è $W \leftarrow W + \eta \nabla$. Poiché vogliamo minimizzare l'errore (andare contro il gradiente), usiamo un η negativo. È equivalente a scrivere $W \leftarrow W - |\eta| \nabla$.

Pesi Iniziali (Ricostruiti dai calcoli intermedi della soluzione):

- **Input $\rightarrow$ Hidden ($W^{(1)}$):**

$$W^{(1)} = \begin{bmatrix} w_{13} & w_{23} \\ w_{14} & w_{24} \end{bmatrix} = \begin{bmatrix} 1.0 & -3.0 \\ 0.2 & 1.0 \end{bmatrix}$$

(Dove la riga indica il nodo di destinazione e la colonna il nodo di partenza).

- **Hidden $\rightarrow$ Output ($W^{(2)}$):**

$$W^{(2)} = \begin{bmatrix} w_{35} & w_{45} \end{bmatrix} = \begin{bmatrix} 1.0 & -1.0 \end{bmatrix}$$

4.8.2 2. Forward Pass (Calcolo delle Previsioni)

Step A: Calcolo ingressi netti allo strato nascosto (net)

$$net_3 = w_{13}x_1 + w_{23}x_2 = (1.0)(1.0) + (-3.0)(0.1) = 1.0 - 0.3 = \mathbf{0.7}$$

$$net_4 = w_{14}x_1 + w_{24}x_2 = (0.2)(1.0) + (1.0)(0.1) = 0.2 + 0.1 = \mathbf{0.3}$$

Step B: Attivazione Hidden Layer (out) Usiamo la funzione $\tanh(z)$.

$$out_3 = \tanh(0.7) \approx \mathbf{0.604}$$

$$out_4 = \tanh(0.3) \approx \mathbf{0.291}$$

Step C: Calcolo Output Finale Lo strato di output è lineare: $out_5 = \sum w_{i5} \cdot out_i$.

$$\hat{y} = w_{35} \cdot out_3 + w_{45} \cdot out_4$$

$$\hat{y} = (1.0)(0.604) + (-1.0)(0.291) = 0.604 - 0.291 = \mathbf{0.313}$$

Step D: Calcolo della Loss

$$\mathcal{L} = \frac{1}{2}(0.313 - 0.5)^2 = \frac{1}{2}(-0.187)^2 \approx \mathbf{0.0175}$$

4.8.3 3. Backward Pass (Calcolo dei Gradienti)

Dobbiamo calcolare come varia l'errore al variare dei pesi. Usiamo la Chain Rule partendo dalla fine.

Gradiente Output (δ_5)

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = (\hat{y} - y) = 0.313 - 0.5 = \mathbf{-0.187}$$

Gradienti Hidden Layer (δ_3, δ_4) Dobbiamo retro-propagare l'errore attraverso i pesi $W^{(2)}$ e attraverso la derivata della $\tanh$. La derivata di $\tanh(z)$ è $(1 - \tanh^2(z)) = (1 - out^2)$.

1. **Nodo 3:**

$$\frac{\partial \mathcal{L}}{\partial out_3} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot w_{35} = (-0.187) \cdot 1.0 = -0.187$$

$$\delta_3 = \frac{\partial \mathcal{L}}{\partial net_3} = \frac{\partial \mathcal{L}}{\partial out_3} \cdot (1 - out_3^2)$$

$$\delta_3 = (-0.187) \cdot (1 - 0.604^2) = (-0.187) \cdot (1 - 0.365) = -0.187 \cdot 0.635 \approx -\mathbf{0.1187}$$

2. **Nodo 4:**

$$\frac{\partial \mathcal{L}}{\partial out_4} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot w_{45} = (-0.187) \cdot (-1.0) = +0.187$$

$$\delta_4 = \frac{\partial \mathcal{L}}{\partial net_4} = \frac{\partial \mathcal{L}}{\partial out_4} \cdot (1 - out_4^2)$$

$$\delta_4 = (0.187) \cdot (1 - 0.291^2) = (0.187) \cdot (1 - 0.085) = 0.187 \cdot 0.915 \approx +\mathbf{0.1711}$$

4.8.4 4. Calcolo Gradienti dei Pesi e Aggiornamento

Il gradiente per un peso w_{ij} (da j a i) è: Errore a valle (δ_i) $\times$ Input a monte ($input_j$).

A. Pesi Hidden $\rightarrow$ Output ($W^{(2)}$)

- $\nabla w_{35} = \delta_5 \cdot out_3 = (-0.187) \cdot 0.604 \approx -\mathbf{0.113}$

- $\nabla w_{45} = \delta_5 \cdot out_4 = (-0.187) \cdot 0.291 \approx -\mathbf{0.054}$

Aggiornamento (con $\eta = -0.2$):

$$w_{35}^{new} = 1.0 + (-0.2)(-0.113) = 1.0 + 0.0226 = \mathbf{1.0226}$$

$$w_{45}^{new} = -1.0 + (-0.2)(-0.054) = -1.0 + 0.0108 = -\mathbf{0.9892}$$

B. Pesi Input $\rightarrow$ Hidden ($W^{(1)}$)

- $\nabla w_{13} = \delta_3 \cdot x_1 = (-0.1187) \cdot 1.0 = -\mathbf{0.1187}$

- $\nabla w_{23} = \delta_3 \cdot x_2 = (-0.1187) \cdot 0.1 \approx -\mathbf{0.0119}$

- $\nabla w_{14} = \delta_4 \cdot x_1 = (0.1711) \cdot 1.0 = \mathbf{0.1711}$

- $\nabla w_{24} = \delta_4 \cdot x_2 = (0.1711) \cdot 0.1 \approx \mathbf{0.0171}$

Aggiornamento (con $\eta = -0.2$):

$$w_{13}^{new} = 1.0 + (-0.2)(-0.1187) = 1.0 + 0.0237 = \mathbf{1.0237}$$

$$w_{23}^{new} = -3.0 + (-0.2)(-0.0119) = -3.0 + 0.0024 = -\mathbf{2.9976}$$

$$w_{14}^{new} = 0.2 + (-0.2)(0.1711) = 0.2 - 0.0342 = \mathbf{0.1658}$$

$$w_{24}^{new} = 1.0 + (-0.2)(0.0171) = 1.0 - 0.0034 = \mathbf{0.9966}$$